

COLLEGE OF ENGINEERING

UNIVERSITI TEKNOLOGI MARA

SHAH ALAM

BACHELOR OF ELECTRONIC ENGINEERING (HONS.)

DATA STRUCTURES AND ALGORITHMS (ECE532)

MINI PROJECT REPORT:
INTERNET OF THINGS - BASED ON SMART HOME

PREPARED BY:

CEEE2225TA (GROUP 2)

1. INTAN NURFARAHIN BINTI MOHD SAFINI (2022478396)
2. UNGKU AISHAH NABILA BINTI UNGKU SALLEHUDIN (2022463936)
3. ANIS SUHANA BINTI MOHD AZMI (2023816852)

PREPARED FOR:

TS. DR. ROSLINA BINTI MOHAMAD

1.INTRODUCTION

Nowadays, there is an increase in technology for the development of smart home systems. In the past, to control devices at home, owners had to do it manually which often required a lot of time and effort. However, with the advancement of technology, smart home systems now allow users to control various devices in their homes more easily and efficiently. These technologies help increase human productivity by automating daily tasks such as lighting control, temperature, security, and others. A smart home is an example of an environment that combines various smart devices that function dynamically and can be monitored and controlled remotely. By using efficient data structures, such as linked lists, trees, and graphs, this system makes the data management process smoother, more organized, and easier to manage. This allows users to monitor and control existing devices more efficiently and makes daily life easier and safer.

1.1 PROBLEM STATEMENT

In this modern world, most homes have invested in smart IoT devices to ease everyday routines. However, too many Iot devices installed have come with several challenges. One of the challenges is manually managing a large number of interconnected devices without a proper system to add, remove or arrange the devices as it can lead to disorganized data. This may lead to difficulty for users to keep track of all the devices. Apart from that, in a disorganized dataset, searching for certain devices or readings can be time consuming and ineffective as the number of devices increases.

Next, inefficient device routing in smart homes can lead to delay to the system performance. Without the presence of algorithms to optimize the path, the network's communication efficiency will decrease particularly in systems with multiple nodes. In a nutshell, the rapid adaptation of IoT devices needs systematic solutions to ensure efficient operations.

1.2 OBJECTIVE

The purposes of this mini project called IoT Based on Smart Home Monitoring System are:

- To develop a database system to add, remove, and sort all the smart home devices that have been installed using a linked list.
- To find sensors based on the type and the reading of the devices using binary search trees.
- To calculate the shortest path of the device's connection by using Kruskal, Prim, and Djikstra algorithms and find alternative paths by using breadth-first search and depth-first search.

1.3 SCOPE PROJECT

The scope of this project is to develop a programming system that has data structures in linked lists, trees, and graphs aimed at solving engineering problems. This project has three main tasks that need to be completed. For the first, insert and store data in linked lists, including adding, deleting, and using sorting algorithms. The second task is to build a tree based on the input data and use three traversal and search algorithms. The third task is to build a graph and implement the search/traversal algorithm and three Greedy methods. In addition, this system will also use hashing algorithms for data verification processes, ensuring security and reliability in information management. This project focuses on the use of appropriate data structures and the application of the best algorithms to solve complex problems in engineering applications.

2. FLOWCHART

2.1. Main Flowchart

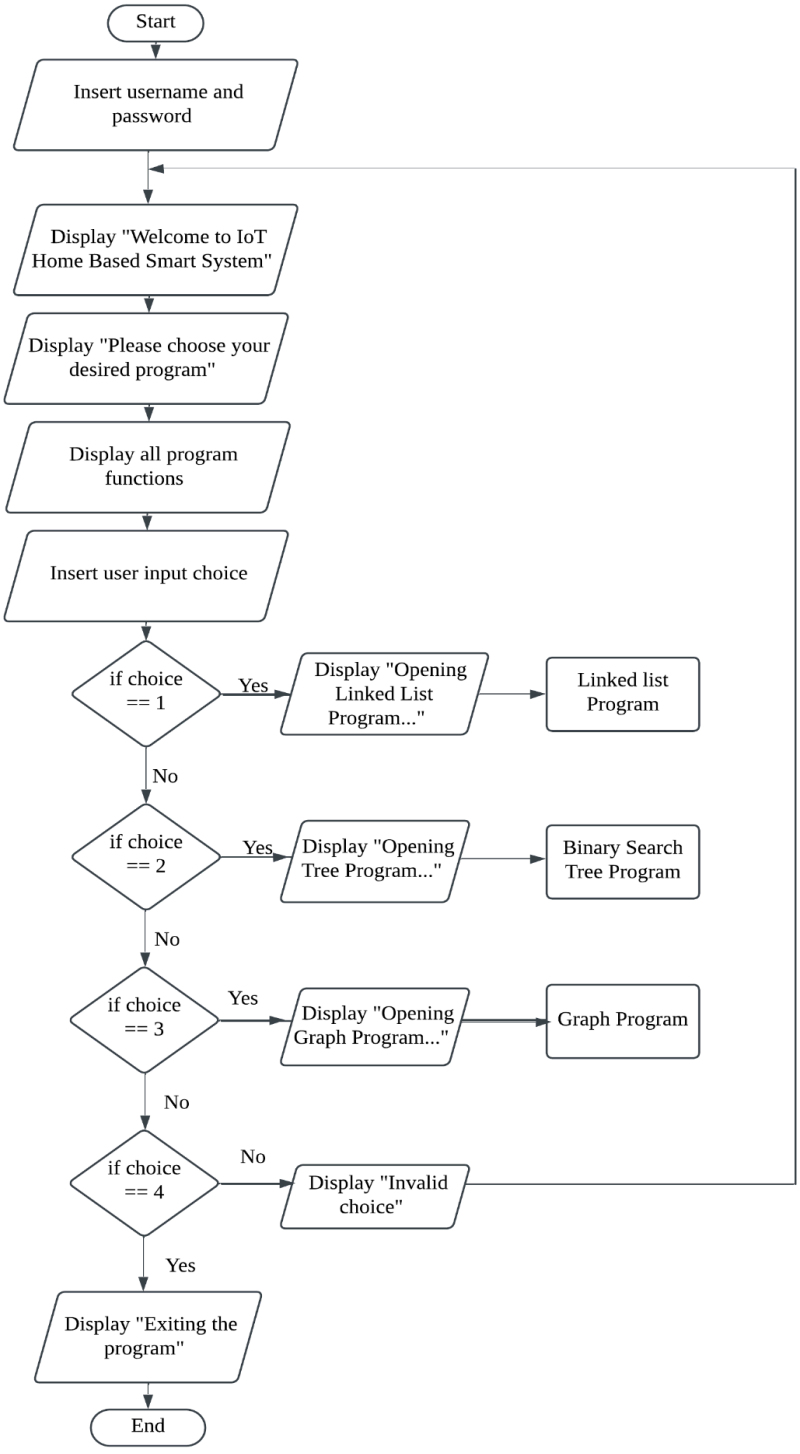


Figure 2.1 shows the flowchart of the main program

This hashing page will first appear when the program starts to run. Users will need to enter username and password first, then the program will display “Welcome to IoT Home Based Smart System” along with four other options. When the user first chooses input ‘1’, the program will display “Opening Linked List Program..” and it will take the user to the linked list menu. If the user chooses input ‘’2’, the program will display “Opening Tree Program…” and a binary search tree menu will appear. Next, when the user chooses input ‘3’, the program will display “Opening Graph Program…” and the user will be directed to the graph menu. When the user chooses input ‘4’, it will display “Exiting the program” and it will end the program. However, if the user chooses a number from 1 to 4, the program will display “invalid” and the user will be taken back to the main page until the user puts the correct input.

2.2 Linked List Flowchart

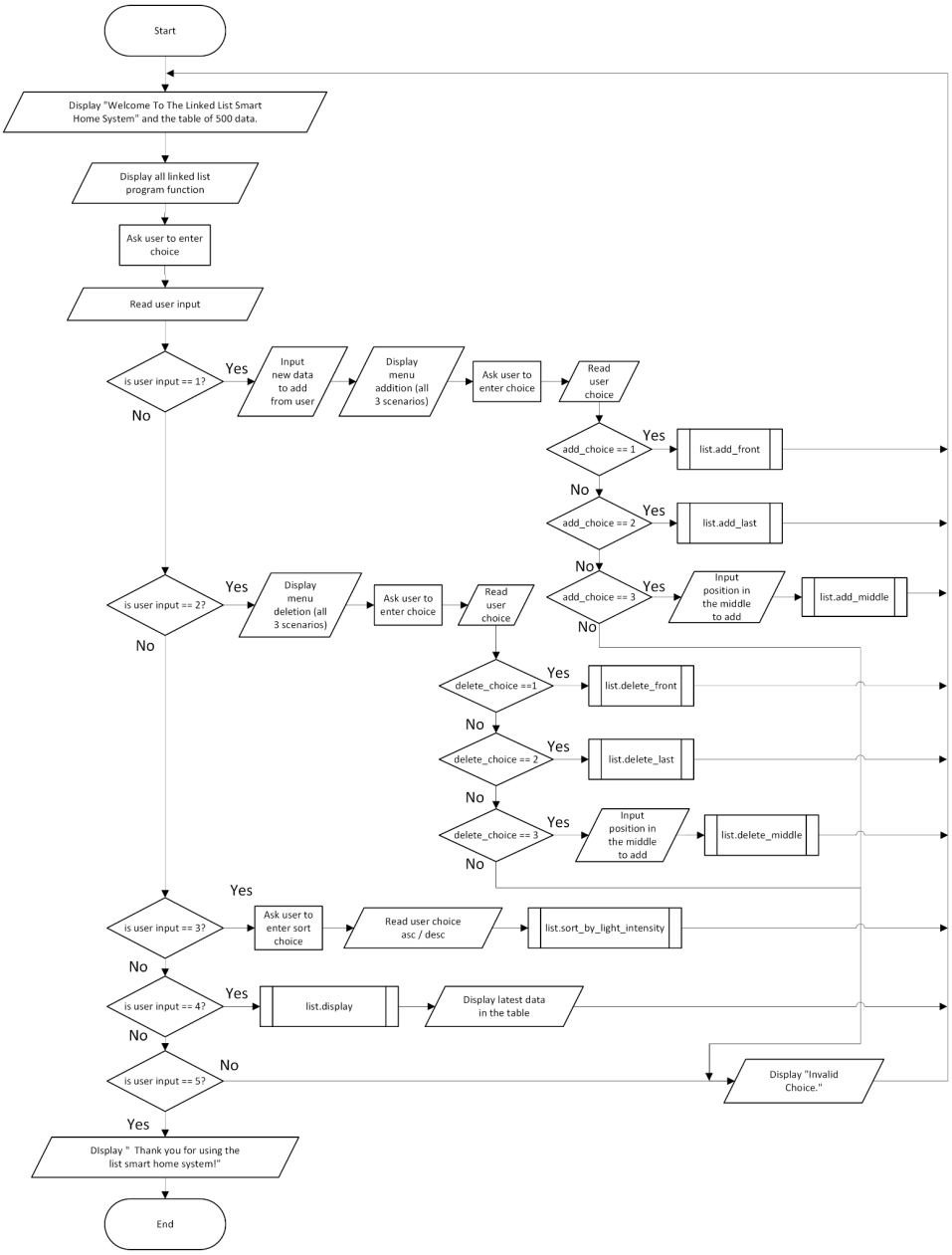


Figure 2.2 shows the flowchart of the linked list program

Figure 2.2 shows the flowchart of the linked list program after the main program was run. In the early program which is the main menu of the system, when the user inputs '1' in the main menu program which is an option for the linked list system, the program will automatically jump into the file of Linked_ list.py to run the program of linked list smart home system. So here is the flowchart that contains four processes the user can choose to better manage the

system of the smart home. The data that need this linked list service is data of light intensity and light location for smart home systems. First thing after choosing the linked list system, the program will display “Welcome to the Linked List Smart Home System” along with 500 data light intensity and light location in the table. Also, the program will display all available options that the user can choose based on the program of the linked list system. There are 5 options processes that users can choose which are adding new data that has 3 scenarios, deletion data that also has 3 scenarios, sorting by data of light intensity, displaying the latest table of data in the linked list system, and exiting the linked list system.

Firstly, when the user inserts input ‘1’ the user will be asked to key in the two new data to add which are data of light intensity and light location and the program will display the 3 option addition (3 scenarios) which is add data at the first list of data, add data at the last list of data and add data in the middle list of the data. After the user makes a choice, which is key in the choice ‘1’, the program will go to the list function.add_data, while for the choice ‘2’ will go to the function of list.add_data, and for choice ‘3’ the program will ask the user to key in the specific position in the list to add data and then go to the function of list.add_middle. Secondly, when the user inserts the input ‘2’, the program will display the menu of deletion (3 scenarios) which is delete data at the first list of data, delete data at the last list of data, and delete data in the middle list of the data. After the user makes a choice, which is key in the choice ‘1’, the program will go to the list function.delete_data, while for choice ‘2’ will go to the function of list.delete_data, and for choice ‘3’ the program will ask the user to key in the specific position in the list to delete data and then go to the function of list.delete_middle.

Besides, if the user inserts input ‘3’, the users need to enter either ‘asc’ for ascending or ‘desc’ for descending and then the program will go to the function list.sort_by_light_intensity. In addition, if the user inserts input ‘4’, the program will go to the function list. Display and directly display the latest data table. The last option process is when the user inserts input ‘5’, then the program will display “Thank you for using the list smart home system!” to exit the program of the linked list and go to the main menu of the smart home system. Note that, after the user chooses between all 4 options of the linked list process and when the user chooses the wrong option, the program will return to the main menu of the linked list system.

2.3 Binary Search Tree Flowchart

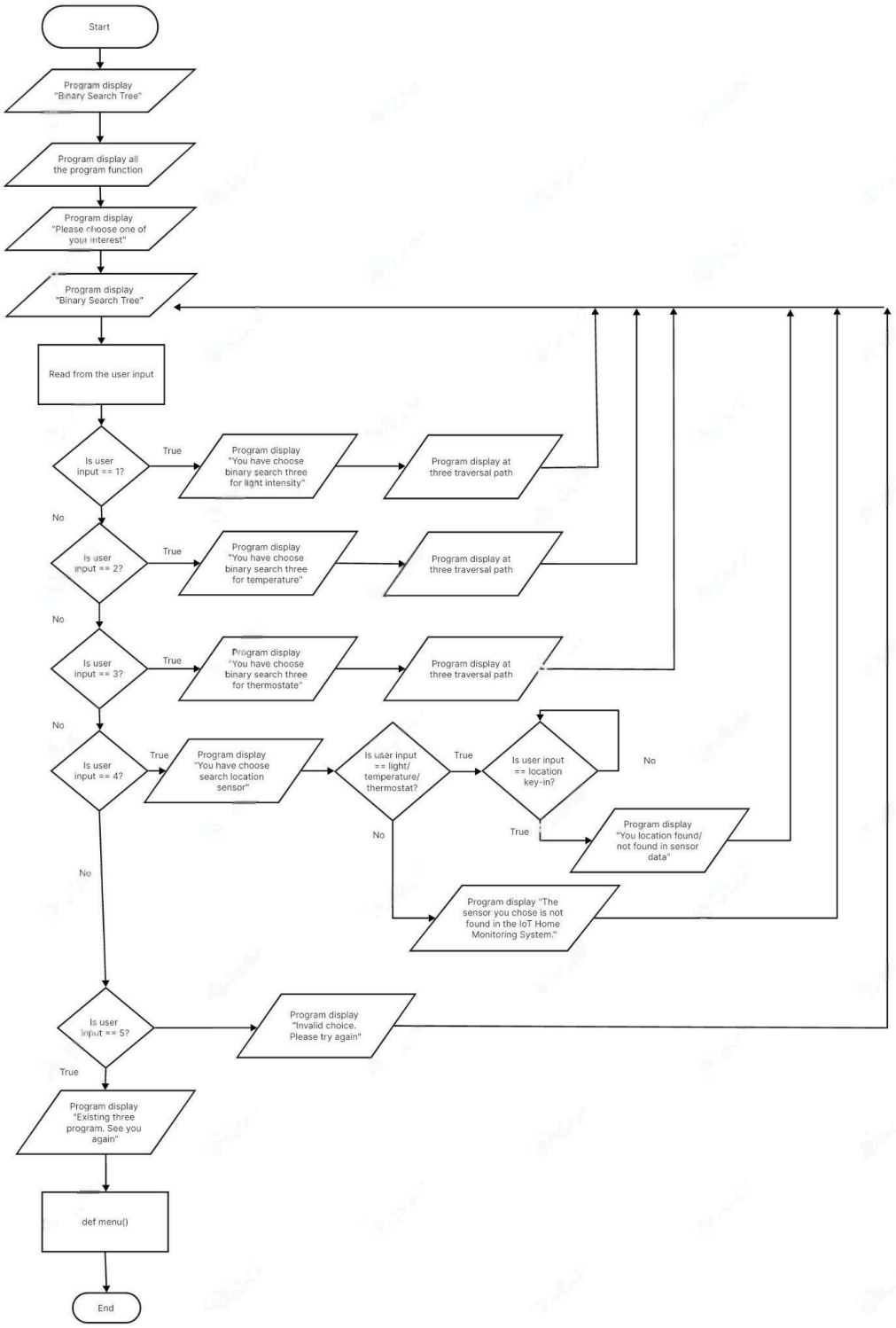


Figure 2.3 shows the flowchart of the tree program

Figure 2.3, shows the flowchart of the binary search tree in this program. In the early program, when user input is ‘2’ in the main menu which indicates the binary search tree option, the program will directly jump to the next file that contains the binary search tree function. In the tree fill, the program will display all the available options that the user can choose based on the program. There are 5 options that users can choose which check the binary search tree by using data of the light intensity, temperature, thermostat and location for light sensor, temperature sensor and thermostat sensor.

By understanding those three different traversal paths, the result will also be different from each traversal path, the result will also be different from each traversal. Therefore, if the user’s input is ‘1’, the program will display all the light intensity that is being used to create a tree and all three traversal results and back to the main menu tree. If the user’s input is ‘2’, the program will show all the data that is being used to create a tree from temperature and will automatically show all the results for three traversals and back to the main menu of the tree. It is the same flow when the user's input is ’3’ where the program will show all the data of the thermostat and the result of the three traversals and back to the main menu tree. Besides, if the user’s input is ‘4’, the users need to enter the type of sensor either light, temperature and thermostat. Then, the users need to enter the location and back to the main menu tree. If the user inserts the type of the sensor other than light, temperature and thermostat, the program will display “The sensor you chose is not found in the Iot Home Monitoring System”. If the user forgot or did not insert the location, the program will ask the user to insert the location again. Lastly, if the user’s input is ‘5’, the program will display “Existing tree program. See you again!” and back to the main menu. If the user's input is then the option, the program will display “Invalid choice. Please try again.” and back to the main menu tree.

2.4 Graph Flowchart

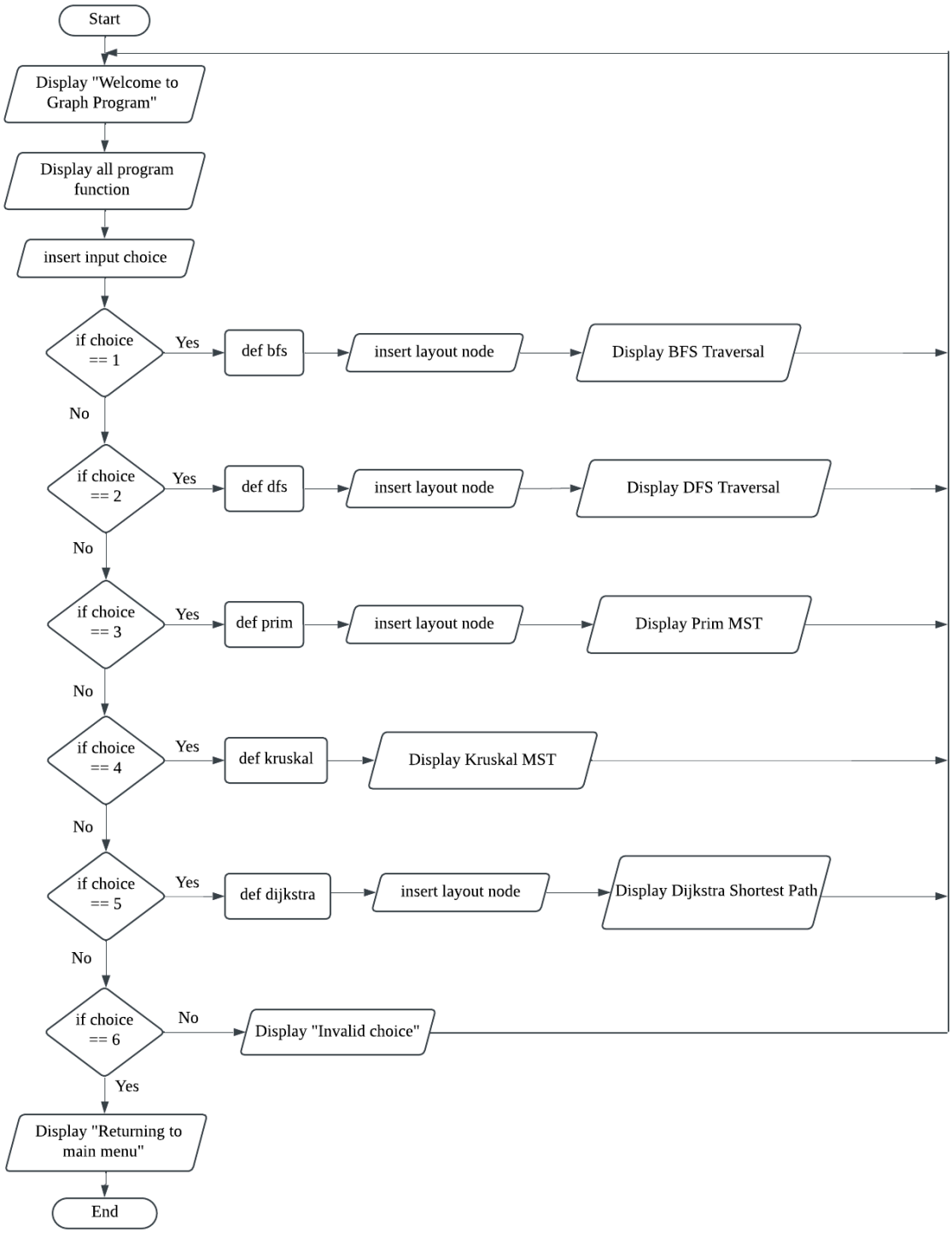


Figure 2.4 shows the flowchart of the graph program

By referring to the main program, when the user chooses ‘3’, the user will be directed to the graph program. The first thing to display will be “Welcome to the Graph Program” along with options of graph algorithms. Users will be asked to enter options again. When the user inserts input ‘1’, the user will be asked to fill the node number as the initial and the program will display the sequence path of breadth first search. Secondly, when the user inserts input ‘2’, the user will once again be asked to insert node number as the starting node and the program will display the result for depth first search.

Next, when the user chooses input ‘3’, the user will once again be needed to enter the node number before the program displays the result for Prim’s algorithm. Furthermore, when the user inserts input ‘4’, the program will be executed right away and produce the result for Kruskal’s algorithm. With input user ‘5’, the program will once again require the user to enter node number as initial vertex before it executes the program and provide the shortest path for dijkstra's algorithm.

Lastly, when the user chooses input ‘6’, the program will display “Returning to main menu” and the program will return to the main program. However, when the user inserts input that is not in between 1 until 6, the program will display “Invalid choice. Please try again.” and the user will be taken back to the graph menu.

3. METHODOLOGY (TASK 1-3)

In order to get the mini project done, a few methods have been used to achieve the objectives of the mini project.At the beginning of the program, this smart home system has security before logging in. Among the methods used is the hashing method with `hashlib.sha256` to ensure that passwords are stored in a secure form. The authentication function allows up to two login attempts, where the username and password input are hashed and compared to the database. This method is user-friendly with clear and flexible guidance messages.

First of all, for the linked list system, the methodology for creating a linked list system in Python that has been used is to define two main classes, namely 'Node' and 'LinkedList'. The 'Node' class serves as the basis for building a linked list. The Node class is essential for creating a linked list. It acts as the basic unit that stores data (Light_Intensity and Light_Location) and a reference to the next node in the list. The Node class is defined by representing an element in a linked list. Each node has two main variables for a linked list: Light_Intensity and Light_Location. The next variable points to the next node in the list. In simple terms, a linked list consists of nodes. Each node contains an element, and each node is linked to its next neighbor. Thus a node can be defined as a class, as follows in the figure:

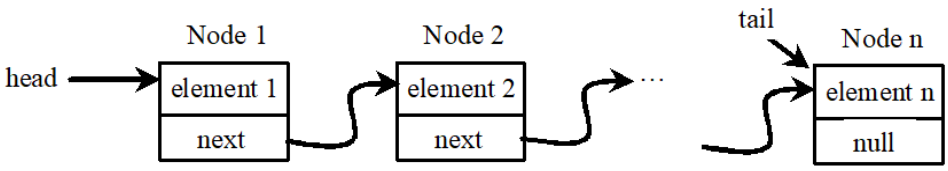


Figure 3.1 shows the linked list structure

Next, the method used is creating the LinkedList class. So the LinkedList class manages the linked list and has a head variable that points to the first node in the list. This LinkedList class also has another 4 methods in it to make the system a more incredible process that users need. The LinkedList class has methods for adding, deleting, sorting, and displaying nodes implemented within this class. For the method of adding nodes, it inserts a new node at the

beginning, puts in a new node at the end, and inserts a node at a specific position in the list. Next, for the method deleting nodes, it removes the first node, and the last node, and removes a node at the specific position in the list. For the sorting nodes by light intensity, the method that is used for sorting in ascending and descending order is bubble sort. The algorithm works by iterating through the linked list multiple times and swapping adjacent nodes if they are out of order. The 'swapped' is used to show whether any swaps occurred during the process. If no swaps were made, the list is considered sorted, and the loop of process swaps is terminated. This is the main feature of the bubble sort, where adjacent elements are repeatedly compared and swapped if they are in the wrong order. The features of bubble sort that can be seen in the code are shown in the figure:

```
# Sort the linked list by Light_Intensity
def sort_by_light_intensity(self, ascending=True):  1 usage
    if self.head is None or self.head.next is None:
        return
    swapped = True
    while swapped:
        swapped = False
        current = self.head
        while current.next:
            if (ascending and current.Light_Intensity > current.next.Light_Intensity) or \
               (not ascending and current.Light_Intensity < current.next.Light_Intensity):
                current.Light_Intensity, current.next.Light_Intensity = current.next.Light_Intensity, current.Light_Intensity
                current.Light_Location, current.next.Light_Location = current.next.Light_Location, current.Light_Location
                swapped = True
            current = current.next
```

Figure 3.2 shows the features of bubble sort in the code.

- Adjacent comparison and swap: 'current.Light_Intensity > current. Next.Light_Intensity'
- Iteration through the loop: 'while swapped:'

In the meantime, the display method uses PrettyTable to print the linked list in a table. It visualizes the linked list that uses the display method using the library PrettyTable to present the 500 data in table format. This makes it easy to see and analyze the linked list data. Last but not least, for the method of creating a linked list system, in creating a menu for good user interaction. The linked_list_menu function provides a user-friendly interface with attractive emojis for interacting with the linked list. The menu provides options to add, delete, sort, and display nodes, making the system accessible and intuitive for the user.

Secondly, trees are an effective method for organizing datasets and searching for specific devices. In this project, a Binary Search Tree (BST) is implemented to manage sensor data efficiently. Each node in the BST contains attributes left, right, and val. The BST is initialized with the root node set to the first element of the dataset. For subsequent elements, if the value is greater than the root's value, it is recursively inserted into the right subtree. Conversely, if the value is less than the root's value, it is inserted into the left subtree. This structure ensures efficient navigation and organization of the dataset. To traverse the BST, three methods are implemented: in-order, pre-order, and post-order traversal. In-order traversal, recursively visits the left subtree, the root, and then the right subtree. This returns elements in ascending order. Pre-order traversal, recursively visits the root, the left subtree, and then the right subtree. Post-order-order traversal, recursively visits the left subtree, the right subtree, and then the roots. When users try to search for a location, the program will build the tree then do the searching. If the root is None or the key matches the root's value, the root is returned. If the key is greater than the root's value, the search continues in the right subtree. If the key is less than the root's value, the search proceeds in the left subtree. This approach ensures efficient and accurate retrieval of the location of the sensor data.

Last but not least, graph is the most efficient method to use to help solving network issues as it represents nodes and edges connection. Graph algorithms can be divided into five sub methods. Breadth-first search (BFS) is ideal usage to explore shortest connection with unweighted networks. On the other hand, depth-first search works by digging through one path deeply before backtracking to another path and the process is repeated until all nodes are visited. As for finding the minimal cost networks, Prim's algorithm is used to connect the smallest connection and avoid any cycle edges. Same goes to Prim's, Kruskal also will avoid cycles in the connection but Kruskal's algorithm is used by prioritizing the smallest connections amongst all. Lastly, to find the shortest path amongst all the connections, Dijkstra's algorithm is used by finding the least weight path.

4.RESULT AND DISCUSSION (OUTPUT AND EXPLANATION) (TASK 1-3)

4.1 MAIN MENU (SHOW OUTPUT WHEN USER CHOOSE 1 AND 2)

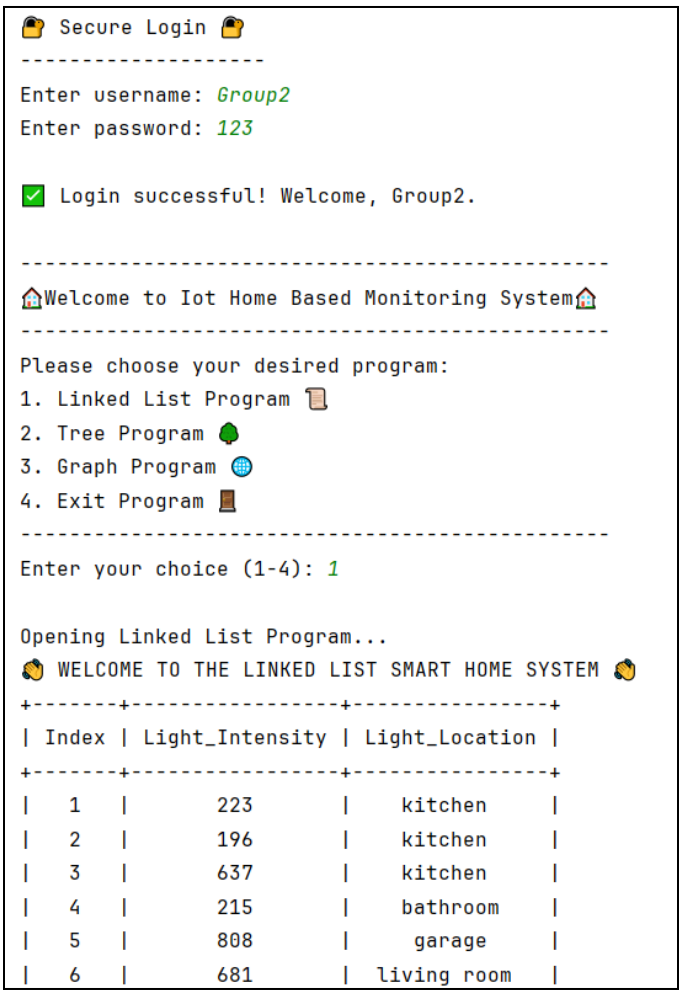


Figure 4.1.1 shows the display when the user inserts option 1.

Based on the output, after entering the username and password for the security system. The program has displayed the main menu. Then when the user chooses input 1, the program will display “Opening Linked List Program…” and users will be directed to 500 data tables and linked list menus.

```
------------------------------------------------
🏠Welcome to Iot Home Based Monitoring System🏠
------------------------------------------------
Please choose your desired program:
1. Linked List Program 📜
2. Tree Program 🌳
3. Graph Program 🌐
4. Exit Program 🚪
------------------------------------------------
Enter your choice (1-4): 2

Opening Tree Program...

--------------------------------------
 🌳 Binary Search Tree Program 🌳
--------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🎚️
[4] Binary search tree for search sensor
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*
```

Figure 4.1.2 shows the display when the user inserts option 2.

Based on the output, when the user chooses input 2, the program will display “Opening Tree Program…” and the user will be directed to the tree menu.

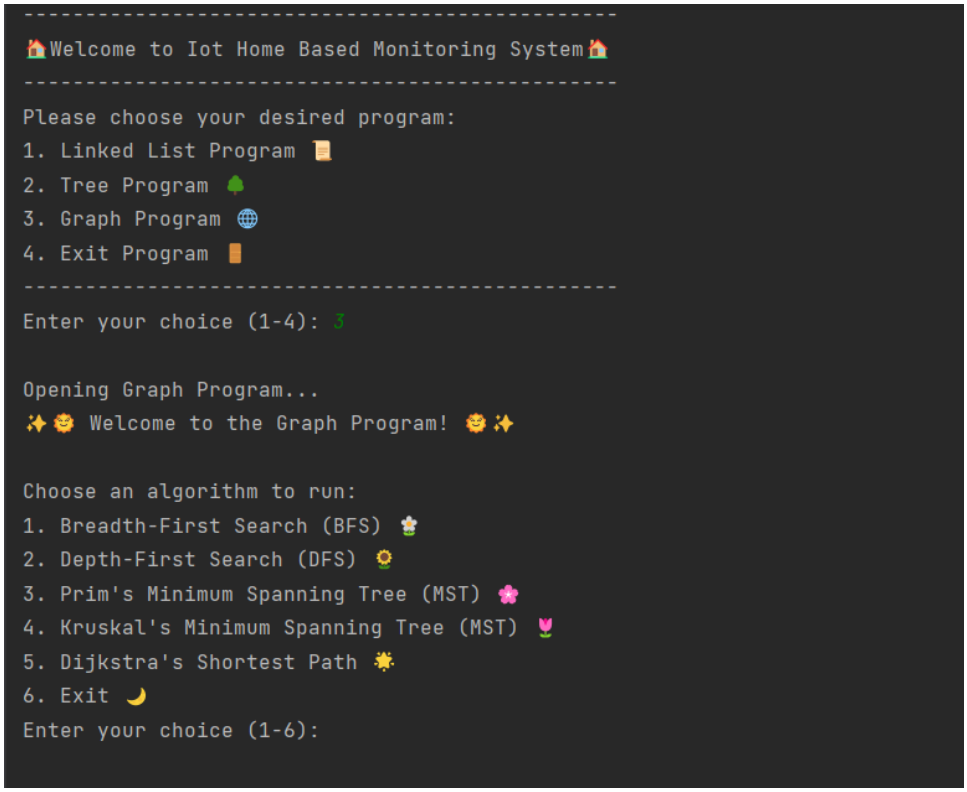


Figure 4.1.3 shows the display when the user inserts option 3.

Based on the output, when the user chooses input 3, the program will display “Opening Graph Program…” and the user will be directed to the graph menu.

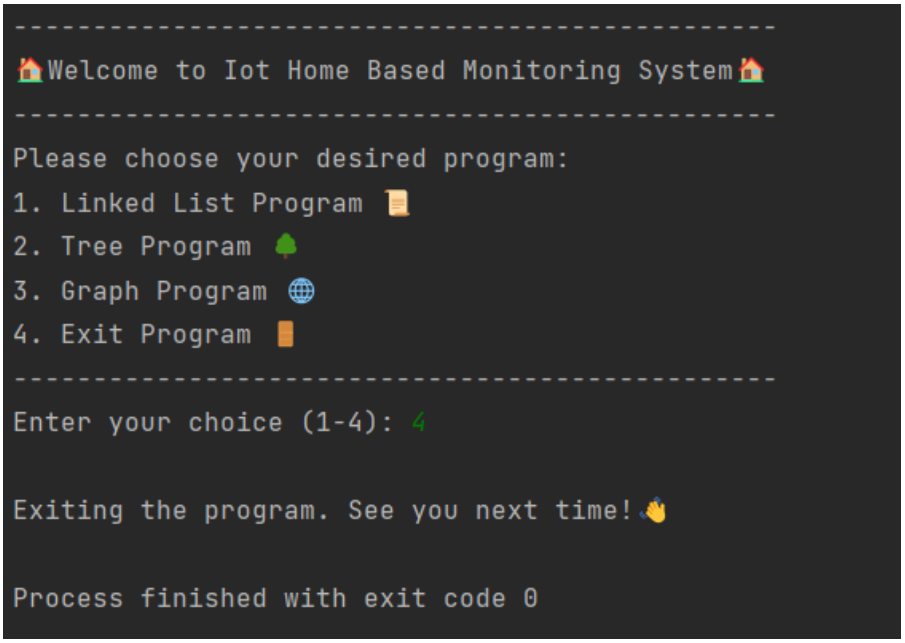


Figure 4.1.4 shows the display when user choose option 4

Based on the figure above, when the user chooses option 4, the program will display "Exiting the program. See you next time!" and the program will end.

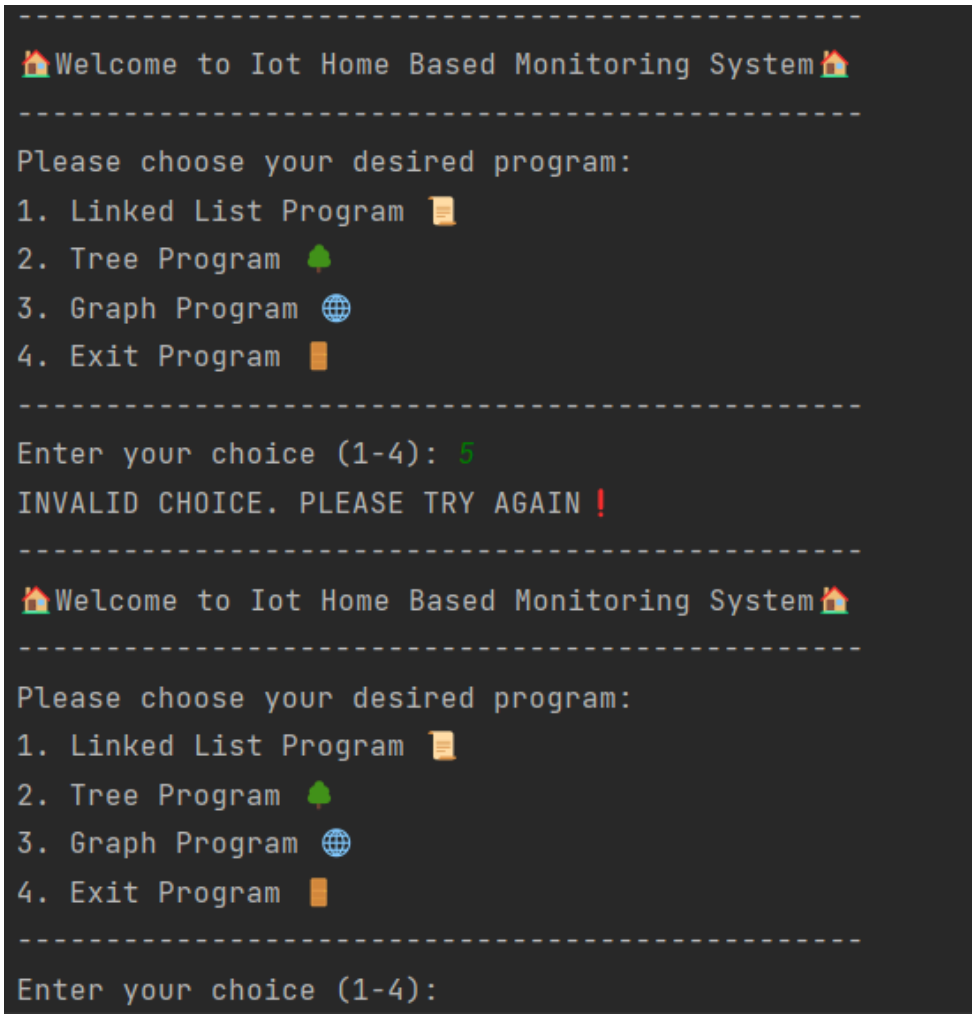


Figure 4.1.5 shows the display when incorrect output is entered

Based on the demo above, when a user inserts a value other than 1 to 4, the program will display "INVALID CHOICE. PLEASE TRY AGAIN" and the user will be taken back to the same menu to enter the correct input value.

4.2 LINKED LIST

For the linked list program, there are 5 different options that the user can choose from. Among the options of this linked list system program are add, delete, sort, display, and exit to the main menu.

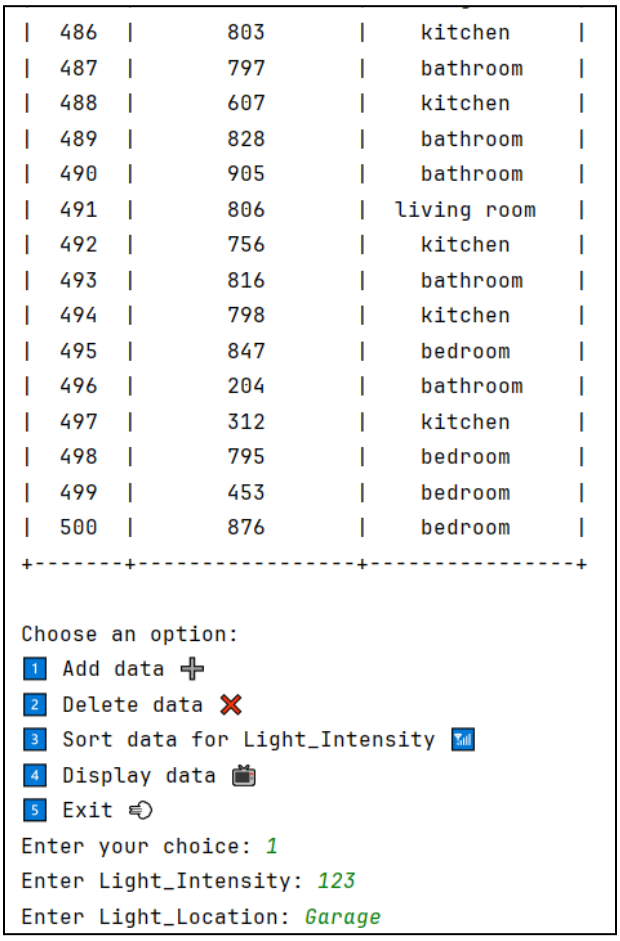


Figure 4.2.1 shows the output display of the linked list menu

When entering the linked list program, this program system will display 500 data in the table first followed by the system options menu for the linked list and if the user chooses option '1', the program asks the user to enter input based on the new light intensity and light location data that the user wants to add.

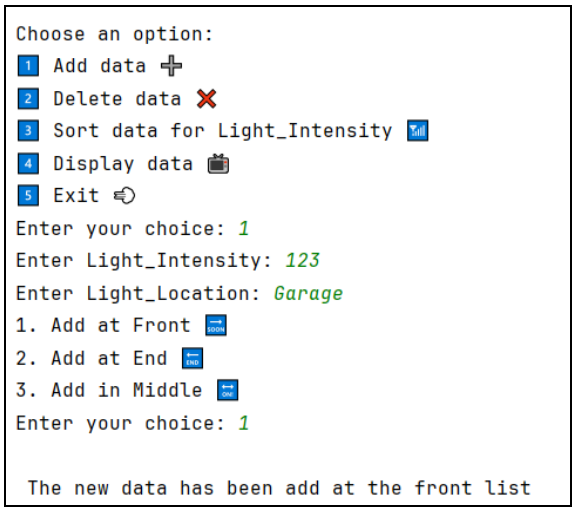


Figure 4.2.2 shows the output display of the option for the addition process at the front list.

After the user keys in new data, the program will display a menu of options for the addition process which has 3 options. If the user selects option '1' the addition process at the front will run while selecting option '2' the addition process at the end will run as well and the program will display the completion of each option.

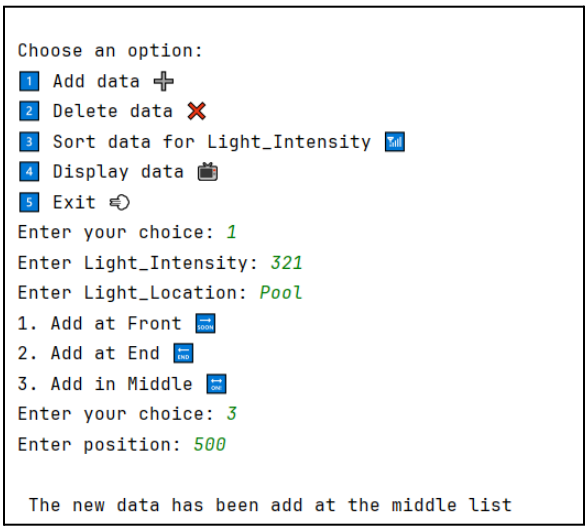


Figure 4.2.3 shows the output display of the option for the addition process in the middle.

When the user chooses option '3' for the addition process, the user will key in what specified position in the middle to add data. After the user key in, the program will display completion as well.

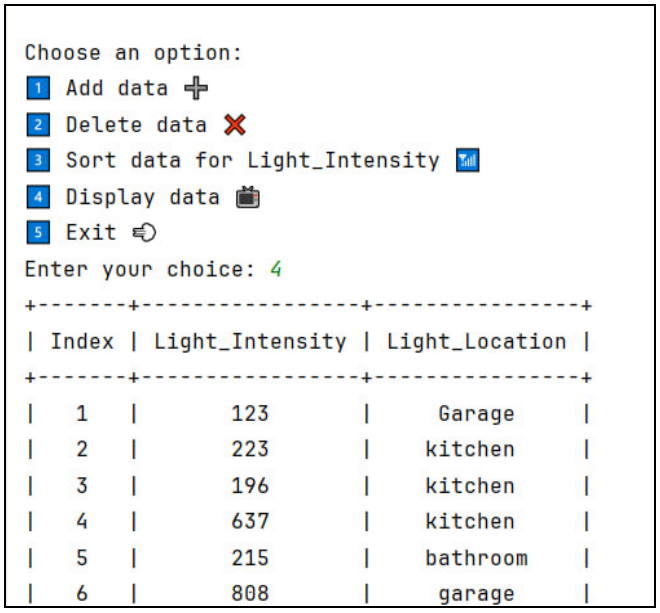


Figure 4.2.4 shows the display output of only the upper part of the data table.

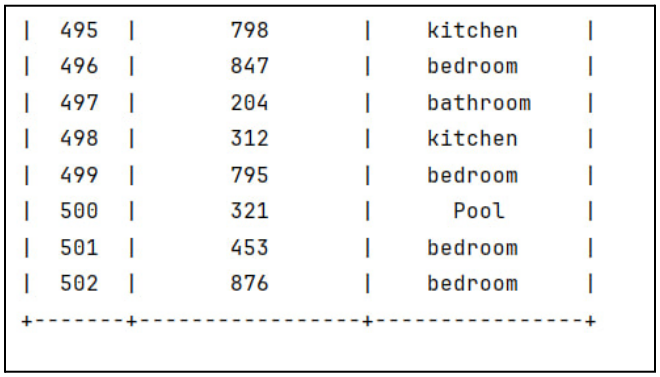


Figure 4.2.5 shows the display output of only the lower part of the data table.

Figures 4.2.4 and 4.2.5 show the top and bottom of the data in the latest table after the user made the choice to add data. It is observed that the data entered by the user has been added to the table. For the data light intensity: 123 and light location: Garage which was added at the front list before has been proven to be added at index no 1, based on figure 4.2.4. For the data light intensity: 321 and light location: Pool which was added to the middle according to the specified position index which is 500 before has been proven to be added. Referring to Figure 4.2.5 the data is already at index 500 (position). In addition, it can be seen that the data has increased to 502 due to the user choosing the addition process 2 times.

```
|  499  |       795       |    bedroom     |
|  500  |       321       |      Pool      |
|  501  |       453       |    bedroom     |
|  502  |       876       |    bedroom     |
+-------+-----------------+----------------+

Choose an option:
1 Add data
2 Delete data
3 Sort data for Light_Intensity
4 Display data
5 Exit
Enter your choice: 2
1. Delete Front
2. Delete End
3. Delete Middle
Enter your choice: 2

 The data has been delete at the last list
```

Figure 4.2.6 shows the output display of the option for the deletion process at the end list.

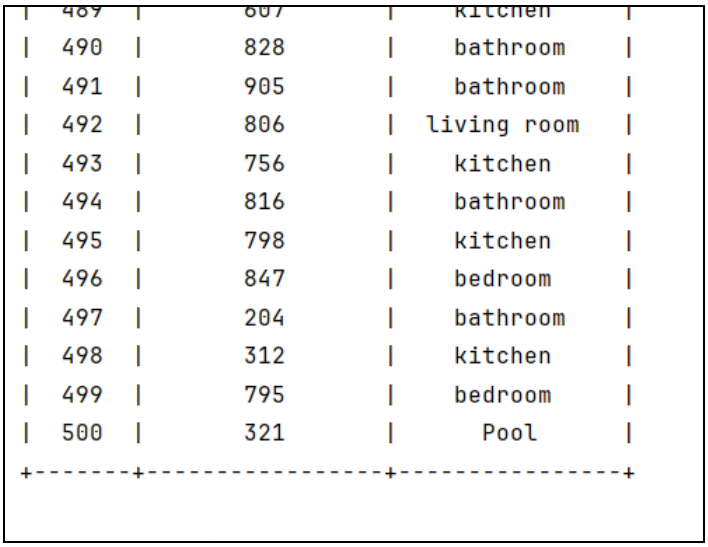
If the user selects option '2' which is process deletion, the program will display a menu of options for the deletion process which has 3 options. If the user selects option '1' the deletion process on the front list will run, while selecting option '2' the deletion process on the end list will also run and the program will display the completion of each option.

```
Choose an option:
1 Add data
2 Delete data
3 Sort data for Light_Intensity
4 Display data
5 Exit
Enter your choice: 2
1. Delete Front
2. Delete End
3. Delete Middle
Enter your choice: 3
Enter position: 501

 The data has been delete at the middle list
```

Figure 4.2.7 shows the output display of the option for the deletion process in the middle.

When the user chooses option '3' for the deletion process, the user will key in what specific position in the middle to delete data. After the user key in, the program will display completion as well.



489	807	kitchen
490	828	bathroom
491	905	bathroom
492	806	living room
493	756	kitchen
494	816	bathroom
495	798	kitchen
496	847	bedroom
497	204	bathroom
498	312	kitchen
499	795	bedroom
500	321	Pool

Figure 4.2.8 shows the display output of only the lower part of the latest data table.

Figure 4.2.8 shows that 2 data have been deleted, namely data index 501 and 502 which were previously in the data table referring to figure 4.2.6. The deletion process has been proven after the user chooses to delete data at the end of the list and delete in the middle of the list with a specific index which is 501. In addition, to get the latest data table list, the user chooses option '4' to display the process for all the latest data in the table perfectly.

```
Choose an option:
1 Add data ➕
2 Delete data ❌
3 Sort data for Light_Intensity 📶
4 Display data 📺
5 Exit 🔙
Enter your choice: 3
Sort order (asc 🅾Ⓡ desc): asc

 The data of light intensity has been sort in list

Choose an option:
1 Add data ➕
2 Delete data ❌
3 Sort data for Light_Intensity 📶
4 Display data 📺
5 Exit 🔙
Enter your choice: 4
+-------+-----------------+----------------+
| Index | Light_Intensity | Light_Location |
+-------+-----------------+----------------+
|   1   |       101       |    kitchen     |
|   2   |       101       |    bedroom     |
|   3   |       102       |    bathroom    |
|   4   |       103       |  living room   |
|   5   |       104       |    kitchen     |
|   6   |       108       |  living room   |
|   7   |       112       |    bedroom     |
|   8   |       114       |     garage     |
```

Figure 4.2.9 shows the display output of the sorting process for ascending order with only the upper part of the latest data table.

Refer to figure 4.2.9 above after the user selects option '3' which is the sorting process for light intensity, and key in the choice for ascending order by entering 'asc'. Then the program displays the completion which is "The data of light intensity has been sorted in the list". To prove that the data has been sorted, the user selects display option '4' as seen in figure 4.2.9 that the light intensity is already in ascending order.

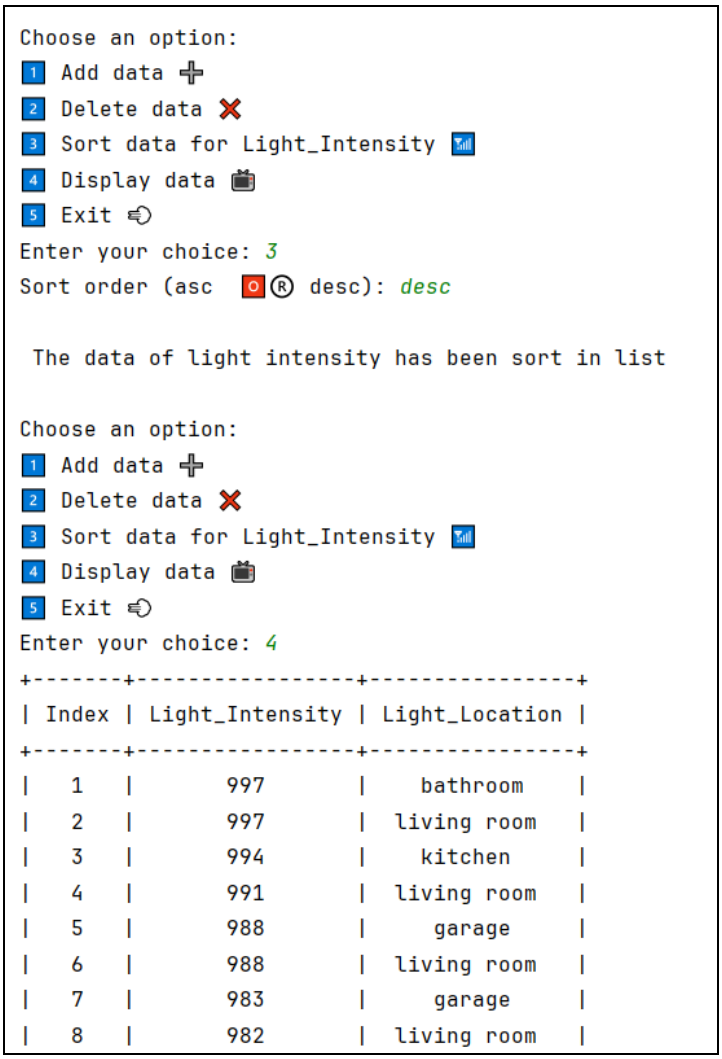


Figure 4.2.10 shows the display output of the sorting process for descending order with only the upper part of the latest data table.

Refer to figure 4.2.10 above after the user selects option '3' which is a sorting process for light intensity data, and key in the choice for descending order by entering 'desc'. Then the program displays the completion which is "The data of light intensity has been sorted in the list". To prove that the data has been sorted, the user selects display option '4' as seen in figure 4.2.10 where the light intensity is already in descending order.

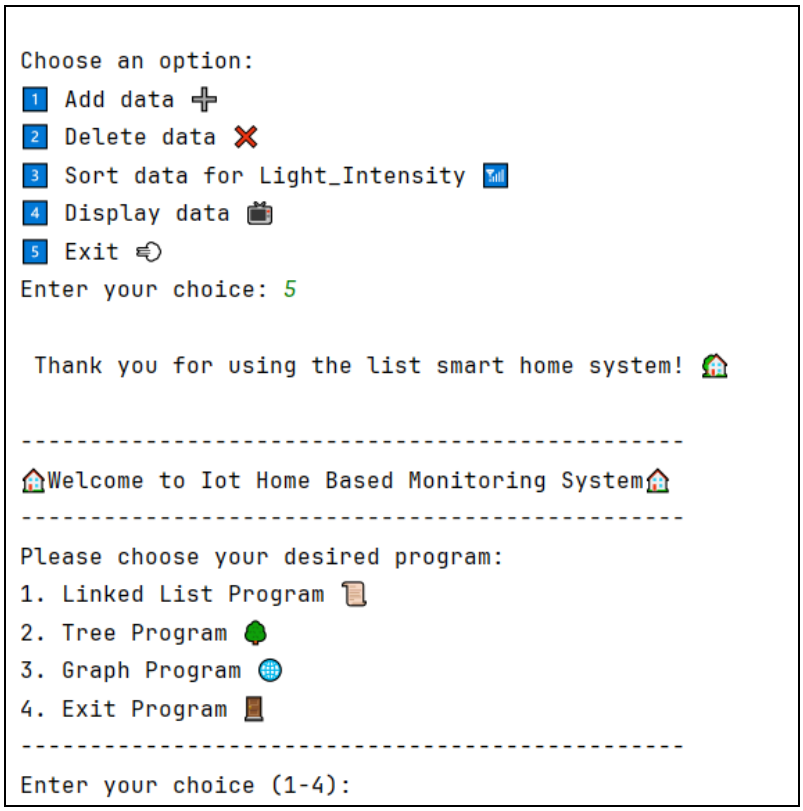


Figure 4.2.11 shows the output display of the main menu smart home system after the linked list system exits.

Referring to Figure 4.2.11 shows the user's choice of option '5' and the program displays completion, which is "Thank you for using the smart home system list! " and the program goes to the main menu then displays 3 program options for the user to choose for the next system process.

4.3 BINARY SEARCH TREE

For the binary search tree program, the program will ask the users to give an input based on their interest. There are 5 different interests that can be run by the program which is to check traversal light intensity, temperature, thermostat and search sensor.

```
---------------------------------------
 🌳 Binary Search Tree Program 🌳
---------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🚹
[4] Binary search tree for search sensor
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*

Enter your choice: 1
You have chosen binary search tree for light intensity.
```

Figure 4.3.1: Main tree program when users choose 1

Firstly, if the user's input is 1, the program will display "You have chosen binary search tree for light intensity" and will show all the result data elements to build in the tree and results of in-order traversal, pre-order traversal and post-order traversal. Next, the program will go back to the main tree program.

```
Building tree with these elements:
 [223, 196, 637, 215, 808, 681, 871, 584, 991, 983, 397, 982, 526, 422, 665, 508, 235, 813, 297, 419, 322, 426, 142, 434, 19
```

Traversal Type	Nodes
In-order	101, 102, 103, 104, 108, 112, 114, 115, 118, 120, 123, 124, 130, 131, 134, 136, 142, 143, 144, 145
	146, 151, 152, 155, 157, 158, 159, 160, 162, 164, 170, 172, 173, 174, 176, 178, 180, 183, 184, 188
	189, 190, 194, 196, 199, 202, 204, 205, 209, 211, 215, 220, 221, 222, 223, 227, 229, 232, 233, 235
	239, 240, 242, 243, 244, 245, 247, 248, 250, 253, 256, 259, 261, 262, 264, 266, 267, 269, 273, 275
	276, 281, 282, 283, 284, 286, 287, 288, 293, 294, 295, 296, 297, 299, 300, 304, 306, 307, 310, 312
	313, 314, 318, 320, 321, 322, 328, 329, 349, 350, 351, 354, 355, 356, 363, 365, 367, 373, 382, 383
	385, 388, 395, 397, 399, 402, 403, 407, 411, 413, 416, 417, 418, 419, 420, 421, 422, 423, 424, 426
	428, 432, 434, 435, 438, 439, 440, 441, 443, 445, 446, 449, 451, 453, 454, 457, 460, 461, 463, 465
	470, 472, 474, 476, 477, 480, 482, 485, 486, 488, 490, 496, 497, 505, 508, 509, 510, 515, 517, 518
	523, 526, 527, 528, 531, 532, 533, 534, 535, 537, 539, 543, 550, 552, 559, 560, 561, 563, 564, 566
	567, 569, 572, 573, 574, 575, 576, 578, 580, 582, 584, 586, 587, 592, 594, 595, 597, 600, 601, 602
	604, 607, 608, 611, 612, 614, 621, 625, 627, 628, 629, 636, 637, 652, 654, 657, 658, 660, 661, 665
	666, 667, 674, 675, 678, 679, 681, 686, 687, 693, 694, 701, 702, 706, 714, 715, 717, 718, 727, 730
	731, 733, 734, 736, 738, 743, 752, 753, 754, 755, 756, 758, 761, 764, 767, 768, 771, 778, 780, 782
	784, 785, 786, 790, 791, 792, 795, 797, 798, 799, 800, 802, 803, 806, 808, 810, 812, 813, 816, 818
	823, 825, 826, 828, 829, 830, 833, 835, 837, 839, 842, 844, 845, 846, 847, 848, 851, 852, 859, 861
	864, 865, 866, 870, 871, 872, 873, 874, 876, 877, 882, 884, 887, 892, 893, 897, 901, 902, 903, 904
	905, 907, 909, 911, 913, 914, 917, 919, 921, 924, 926, 927, 929, 931, 937, 940, 946, 949, 950, 952
	956, 958, 959, 966, 969, 971, 972, 973, 979, 982, 983, 988, 991, 994, 997

Figure 4.3.2: In-order traversal for light intensity

	956, 958, 959, 966, 969, 971, 972, 973, 979, 982, 983, 988, 991, 994, 997
Pre-order	223, 196, 142, 101, 136, 120, 118, 115, 112, 108, 103, 102, 104, 114, 123, 134, 124, 130, 131, 194
	174, 145, 144, 143, 172, 155, 151, 146, 152, 170, 160, 158, 157, 159, 162, 164, 173, 180, 176, 178
	190, 189, 184, 183, 188, 215, 202, 199, 211, 204, 209, 205, 222, 220, 221, 637, 584, 397, 235, 232
	227, 229, 233, 297, 293, 261, 250, 248, 245, 242, 240, 239, 243, 244, 247, 259, 253, 256, 273, 264
	262, 269, 266, 267, 288, 286, 282, 281, 275, 276, 283, 284, 287, 295, 294, 296, 322, 313, 306, 300
	299, 304, 307, 310, 312, 320, 318, 314, 321, 356, 349, 329, 328, 354, 350, 351, 355, 367, 363, 365
	385, 383, 382, 373, 395, 388, 526, 422, 419, 402, 399, 418, 417, 403, 413, 411, 407, 416, 421, 420
	508, 426, 424, 423, 434, 432, 428, 460, 445, 435, 438, 440, 439, 441, 443, 457, 446, 449, 451, 454
	453, 477, 472, 463, 461, 470, 465, 474, 476, 505, 485, 482, 480, 490, 486, 488, 496, 497, 518, 510
	509, 517, 515, 523, 550, 534, 528, 527, 531, 533, 532, 535, 543, 539, 537, 576, 569, 560, 552, 559
	564, 563, 561, 566, 567, 574, 572, 573, 575, 578, 582, 580, 621, 600, 595, 594, 587, 586, 592, 597
	607, 604, 602, 601, 608, 611, 614, 612, 627, 625, 629, 628, 636, 808, 681, 665, 660, 657, 654, 652
	658, 661, 674, 667, 666, 679, 678, 675, 738, 693, 686, 687, 714, 702, 694, 701, 706, 736, 734, 731
	715, 717, 727, 718, 730, 733, 764, 754, 743, 752, 753, 755, 761, 756, 758, 768, 767, 771, 799, 791
	782, 780, 778, 785, 784, 790, 786, 792, 797, 795, 798, 800, 803, 802, 806, 871, 813, 810, 812, 848
	818, 816, 829, 826, 825, 823, 828, 833, 830, 839, 835, 837, 844, 842, 845, 846, 847, 870, 851, 864
	852, 861, 859, 865, 866, 991, 983, 982, 971, 924, 909, 877, 873, 872, 874, 876, 887, 884, 882, 902
	897, 892, 893, 901, 904, 903, 907, 905, 917, 913, 911, 914, 919, 921, 969, 952, 931, 929, 927, 926
	937, 949, 940, 946, 950, 959, 956, 958, 966, 979, 973, 972, 988, 997, 994

Figure 4.3.3: Pre-order traversal for light intensity

Post-order	102, 104, 103, 108, 114, 112, 115, 118, 131, 130, 124, 134, 123, 120, 136, 101, 143, 144, 146, 152
	151, 157, 159, 158, 164, 162, 160, 170, 155, 173, 172, 145, 178, 176, 183, 188, 184, 189, 190, 180
	174, 194, 142, 199, 205, 209, 204, 211, 202, 221, 220, 222, 215, 196, 229, 227, 233, 232, 239, 240
	244, 243, 242, 247, 245, 248, 256, 253, 259, 250, 262, 267, 266, 269, 264, 276, 275, 281, 284, 283
	282, 287, 286, 288, 273, 261, 294, 296, 295, 293, 299, 304, 300, 312, 310, 307, 306, 314, 318, 321
	320, 313, 328, 329, 351, 350, 355, 354, 349, 365, 363, 373, 382, 383, 388, 395, 385, 367, 356, 322
	297, 235, 399, 407, 411, 416, 413, 403, 417, 418, 402, 420, 421, 419, 423, 424, 428, 432, 439, 443
	441, 440, 438, 435, 453, 454, 451, 449, 446, 457, 445, 461, 465, 470, 463, 476, 474, 472, 480, 482
	488, 486, 497, 496, 490, 485, 505, 477, 460, 434, 426, 509, 515, 517, 510, 523, 518, 508, 422, 527
	532, 533, 531, 528, 537, 539, 543, 535, 534, 559, 552, 561, 563, 567, 566, 564, 560, 573, 572, 575
	574, 569, 580, 582, 578, 576, 550, 526, 397, 586, 592, 587, 594, 597, 595, 601, 602, 604, 612, 614
	611, 608, 607, 600, 625, 628, 636, 629, 627, 621, 584, 652, 654, 658, 657, 661, 660, 666, 667, 675
	678, 679, 674, 665, 687, 686, 701, 694, 706, 702, 718, 730, 727, 717, 715, 733, 731, 734, 736, 714
	693, 753, 752, 743, 758, 756, 761, 755, 754, 767, 778, 780, 784, 786, 790, 785, 782, 795, 798, 797
	792, 791, 802, 806, 803, 800, 799, 771, 768, 764, 738, 681, 812, 810, 816, 823, 825, 828, 826, 830
	837, 835, 842, 847, 846, 845, 844, 839, 833, 829, 818, 859, 861, 852, 866, 865, 864, 851, 870, 848
	813, 872, 876, 874, 873, 882, 884, 893, 892, 901, 897, 903, 905, 907, 904, 902, 887, 877, 911, 914
	913, 921, 919, 917, 909, 926, 927, 929, 946, 940, 950, 949, 937, 931, 958, 956, 966, 959, 952, 969
	924, 972, 973, 979, 971, 982, 988, 983, 994, 997, 991, 871, 808, 637, 223

Figure 4.3.4: Post-order traversal for light intensity

Next, if the user's input is 2, the program will display "You have chosen binary search tree for temperature" and will show all the result data elements to build in the tree and results of in-order traversal, pre-order traversal and post-order traversal. Next, the program will go back to the main tree program.

```
---------------------------------------
 🌳 Binary Search Tree Program 🌳
---------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🚹
[4] Binary search tree for search sensor
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*

Enter your choice: 2
You have chosen binary search tree for temperature.
```

Figure 4.3.5: Main tree program when users choose 2

Building tree with these elements:
[25.23, 21.16, 29.32, 33.73, 32.29, 21.94, 26.61, 23.43, 31.4, 21.59, 30.89, 31.95, 34.65, 30.74, 28.82, 34.78, 34.94, 26.79, 34.63, 26.32, 26.51, 30.42, 30.21, 30.49, 33.8

Traversal Type	Nodes
In-order	20.04, 20.15, 20.18, 20.19, 20.2, 20.24, 20.25, 20.3, 20.33, 20.36, 20.41, 20.44, 20.47, 20.48, 20.52, 20.56, 20.57, 20.58, 20.6, 20.61
	20.65, 20.71, 20.76, 20.8, 20.85, 20.86, 20.87, 20.89, 20.9, 20.93, 20.95, 20.99, 21.04, 21.08, 21.12, 21.15, 21.16, 21.18, 21.19, 21.21
	21.26, 21.29, 21.3, 21.31, 21.34, 21.36, 21.38, 21.39, 21.43, 21.44, 21.48, 21.59, 21.68, 21.71, 21.72, 21.74, 21.77, 21.8, 21.82, 21.83
	21.85, 21.87, 21.91, 21.92, 21.94, 21.96, 22.0, 22.04, 22.09, 22.1, 22.22, 22.27, 22.28, 22.39, 22.4, 22.42, 22.6, 22.64, 22.67, 22.73
	22.84, 22.89, 22.92, 22.96, 22.97, 23.12, 23.16, 23.19, 23.24, 23.42, 23.43, 23.45, 23.46, 23.48, 23.51, 23.53, 23.58, 23.66, 23.68, 23.73
	23.74, 23.76, 23.8, 23.81, 23.85, 23.86, 23.88, 23.97, 24.13, 24.16, 24.18, 24.22, 24.26, 24.29, 24.32, 24.35, 24.41, 24.44, 24.45, 24.49
	24.51, 24.61, 24.63, 24.67, 24.69, 24.7, 24.71, 24.72, 24.8, 24.81, 24.86, 24.91, 24.92, 24.97, 25.0, 25.07, 25.09, 25.13, 25.19, 25.23
	25.27, 25.32, 25.36, 25.38, 25.42, 25.43, 25.54, 25.56, 25.58, 25.59, 25.6, 25.64, 25.68, 25.71, 25.72, 25.78, 25.83, 25.88, 25.89, 25.9
	25.91, 25.93, 25.94, 26.02, 26.04, 26.07, 26.12, 26.17, 26.18, 26.22, 26.28, 26.3, 26.32, 26.33, 26.36, 26.37, 26.39, 26.4, 26.44, 26.47
	26.51, 26.54, 26.57, 26.58, 26.59, 26.61, 26.65, 26.71, 26.79, 26.82, 26.83, 26.84, 26.9, 26.97, 27.02, 27.04, 27.12, 27.17, 27.18, 27.23
	27.26, 27.33, 27.43, 27.49, 27.55, 27.56, 27.6, 27.65, 27.67, 27.69, 27.71, 27.75, 27.76, 27.77, 27.79, 27.82, 27.86, 27.88, 27.89, 27.9
	28.01, 28.02, 28.07, 28.16, 28.2, 28.22, 28.23, 28.25, 28.26, 28.42, 28.44, 28.47, 28.55, 28.58, 28.6, 28.62, 28.66, 28.68, 28.82, 28.83
	28.86, 28.88, 28.9, 28.91, 28.93, 28.95, 28.99, 29.0, 29.03, 29.14, 29.15, 29.16, 29.18, 29.23, 29.24, 29.29, 29.3, 29.31, 29.32, 29.39
	29.52, 29.55, 29.57, 29.59, 29.61, 29.62, 29.63, 29.66, 29.7, 29.72, 29.76, 29.79, 29.8, 29.81, 29.84, 29.85, 29.93, 30.02, 30.04, 30.08
	30.15, 30.19, 30.2, 30.21, 30.22, 30.23, 30.26, 30.28, 30.29, 30.31, 30.33, 30.36, 30.41, 30.42, 30.45, 30.47, 30.49, 30.5, 30.53, 30.62
	30.67, 30.72, 30.74, 30.82, 30.83, 30.84, 30.88, 30.89, 30.91, 30.93, 31.06, 31.08, 31.14, 31.15, 31.16, 31.2, 31.22, 31.25, 31.26, 31.29
	31.33, 31.38, 31.4, 31.42, 31.47, 31.5, 31.52, 31.53, 31.56, 31.61, 31.65, 31.67, 31.69, 31.7, 31.76, 31.82, 31.87, 31.91, 31.93, 31.95
	31.97, 32.07, 32.14, 32.15, 32.2, 32.29, 32.3, 32.36, 32.39, 32.41, 32.44, 32.46, 32.56, 32.63, 32.69, 32.71, 32.75, 32.81, 32.83, 32.85
	32.87, 32.95, 32.97, 32.99, 33.0, 33.03, 33.05, 33.09, 33.1, 33.11, 33.22, 33.24, 33.25, 33.33, 33.43, 33.44, 33.47, 33.56, 33.6, 33.62
	33.67, 33.73, 33.77, 33.78, 33.8, 33.83, 33.84, 33.86, 33.87, 33.93, 33.95, 33.98, 34.0, 34.03, 34.06, 34.11, 34.17, 34.18, 34.26, 34.3
	34.38, 34.42, 34.44, 34.49, 34.53, 34.55, 34.59, 34.6, 34.63, 34.65, 34.67, 34.72, 34.73, 34.77, 34.78, 34.82, 34.83, 34.84, 34.91, 34.92
	34.93, 34.94, 34.96, 34.98

Figure 4.3.6: In-order traversal for temperature

Pre-order	25.23, 21.16, 20.93, 20.6, 20.33, 20.18, 20.15, 20.04, 20.19, 20.25, 20.2, 20.24, 20.3, 20.41, 20.36, 20.52, 20.48, 20.47, 20.44, 20.58
	20.56, 20.57, 20.8, 20.76, 20.61, 20.71, 20.65, 20.87, 20.85, 20.86, 20.89, 20.9, 21.08, 20.95, 21.04, 20.99, 21.15, 21.12, 21.94, 21.59
	21.19, 21.18, 21.36, 21.34, 21.3, 21.26, 21.21, 21.29, 21.31, 21.44, 21.43, 21.38, 21.39, 21.48, 21.87, 21.85, 21.77, 21.72, 21.68, 21.71
	21.74, 21.8, 21.83, 21.82, 21.92, 21.91, 23.43, 22.89, 22.39, 22.28, 22.27, 22.0, 21.96, 22.22, 22.09, 22.04, 22.1, 22.73, 22.42, 22.4
	22.67, 22.64, 22.6, 22.84, 23.16, 22.92, 22.96, 22.97, 23.12, 23.42, 23.19, 23.24, 24.35, 24.29, 23.76, 23.74, 23.58, 23.45, 23.51, 23.46
	23.48, 23.53, 23.73, 23.68, 23.66, 23.81, 23.8, 23.85, 24.13, 23.97, 23.88, 23.86, 24.26, 24.16, 24.18, 24.22, 24.32, 24.97, 24.72, 24.44
	24.41, 24.61, 24.51, 24.49, 24.45, 24.69, 24.67, 24.63, 24.71, 24.7, 24.91, 24.8, 24.86, 24.81, 24.92, 25.0, 25.07, 25.19, 25.13, 25.09
	29.32, 26.61, 26.32, 25.64, 25.6, 25.43, 25.27, 25.32, 25.38, 25.36, 25.42, 25.56, 25.54, 25.58, 25.59, 25.94, 25.88, 25.72, 25.71, 25.68
	25.78, 25.83, 25.9, 25.89, 25.91, 25.93, 26.12, 26.07, 26.02, 26.04, 26.22, 26.17, 26.18, 26.3, 26.28, 26.51, 26.47, 26.37, 26.36, 26.33
	26.4, 26.39, 26.44, 26.54, 26.57, 26.58, 26.59, 28.82, 26.79, 26.65, 26.71, 28.6, 27.77, 27.67, 27.33, 26.83, 26.82, 26.84, 27.17, 26.9
	27.04, 27.02, 26.97, 27.12, 27.23, 27.18, 27.26, 27.56, 27.55, 27.43, 27.49, 27.65, 27.6, 27.69, 27.76, 27.71, 27.75, 28.55, 28.47, 28.16
	27.88, 27.86, 27.82, 27.79, 28.07, 28.01, 27.9, 27.89, 28.02, 28.2, 28.44, 28.22, 28.26, 28.23, 28.25, 28.42, 28.58, 28.68, 28.62, 28.66
	28.86, 28.83, 29.15, 28.99, 28.91, 28.88, 28.9, 28.93, 28.95, 29.0, 29.14, 29.03, 29.3, 29.29, 29.24, 29.16, 29.18, 29.23, 29.31, 33.73
	32.29, 31.4, 30.89, 30.74, 30.42, 30.21, 29.62, 29.55, 29.39, 29.52, 29.57, 29.61, 29.59, 29.8, 29.79, 29.7, 29.66, 29.63, 29.76, 29.72
	29.85, 29.84, 29.81, 29.93, 30.2, 30.08, 30.02, 30.04, 30.15, 30.19, 30.31, 30.26, 30.23, 30.22, 30.28, 30.29, 30.36, 30.33, 30.41, 30.49
	30.45, 30.47, 30.5, 30.62, 30.53, 30.72, 30.67, 30.82, 30.83, 30.84, 30.88, 31.25, 31.08, 31.06, 30.91, 30.93, 31.15, 31.14, 31.16, 31.2
	31.22, 31.38, 31.29, 31.26, 31.33, 31.95, 31.61, 31.42, 31.52, 31.5, 31.47, 31.56, 31.53, 31.87, 31.69, 31.65, 31.67, 31.82, 31.7, 31.76
	31.91, 31.93, 31.97, 32.07, 32.2, 32.15, 32.14, 32.81, 32.46, 32.36, 32.3, 32.41, 32.39, 32.44, 32.71, 32.56, 32.69, 32.63, 32.75, 33.11
	33.0, 32.83, 32.87, 32.85, 32.99, 32.95, 32.97, 33.03, 33.1, 33.09, 33.05, 33.56, 33.22, 33.47, 33.24, 33.25, 33.44, 33.33, 33.43, 33.62
	33.6, 33.67, 34.65, 34.63, 33.8, 33.78, 33.77, 34.0, 33.87, 33.84, 33.83, 33.86, 33.98, 33.93, 33.95, 34.44, 34.18, 34.17, 34.06, 34.03
	34.11, 34.42, 34.26, 34.38, 34.3, 34.59, 34.49, 34.55, 34.53, 34.6, 34.78, 34.73, 34.67, 34.72, 34.77, 34.94, 34.91, 34.82, 34.84, 34.83
	34.92, 34.93, 34.96, 34.98

Figure 4.3.7: Pre-order traversal for temperature

Post-order	20.04, 20.15, 20.24, 20.2, 20.3, 20.25, 20.19, 20.18, 20.36, 20.44, 20.47, 20.48, 20.57, 20.56, 20.58, 20.52, 20.41, 20.33, 20.65, 20.71
	20.61, 20.76, 20.86, 20.85, 20.9, 20.89, 20.87, 20.8, 20.6, 20.99, 21.04, 20.95, 21.12, 21.15, 21.08, 20.93, 21.18, 21.21, 21.29, 21.26
	21.31, 21.3, 21.34, 21.39, 21.38, 21.43, 21.48, 21.44, 21.36, 21.19, 21.71, 21.68, 21.74, 21.72, 21.82, 21.83, 21.8, 21.77, 21.85, 21.91
	21.92, 21.87, 21.59, 21.96, 22.04, 22.1, 22.09, 22.22, 22.0, 22.27, 22.28, 22.4, 22.6, 22.64, 22.67, 22.42, 22.84, 22.73, 22.39, 23.12
	22.97, 22.96, 22.92, 23.24, 23.19, 23.42, 23.16, 22.89, 23.48, 23.46, 23.53, 23.51, 23.45, 23.66, 23.68, 23.73, 23.58, 23.74, 23.8, 23.86
	23.88, 23.97, 24.22, 24.18, 24.16, 24.26, 24.13, 23.85, 23.81, 23.76, 24.32, 24.29, 24.41, 24.45, 24.49, 24.51, 24.63, 24.67, 24.7, 24.71
	24.69, 24.61, 24.44, 24.81, 24.86, 24.8, 24.92, 24.91, 24.72, 25.09, 25.13, 25.19, 25.07, 25.0, 24.97, 24.35, 23.43, 21.94, 21.16, 25.36
	25.42, 25.38, 25.32, 25.27, 25.54, 25.59, 25.58, 25.56, 25.43, 25.6, 25.68, 25.71, 25.83, 25.78, 25.72, 25.89, 25.93, 25.91, 25.9, 25.88
	26.04, 26.02, 26.07, 26.18, 26.17, 26.28, 26.3, 26.22, 26.12, 25.94, 25.64, 26.33, 26.36, 26.39, 26.44, 26.4, 26.37, 26.47, 26.59, 26.58
	26.57, 26.54, 26.51, 26.32, 26.71, 26.65, 26.82, 26.97, 27.02, 27.12, 27.04, 26.9, 27.18, 27.26, 27.23, 27.17, 26.84, 26.83, 27.49, 27.43
	27.55, 27.6, 27.65, 27.56, 27.33, 27.75, 27.71, 27.76, 27.69, 27.67, 27.79, 27.82, 27.86, 27.89, 27.9, 28.02, 28.01, 28.07, 27.88, 28.25
	28.23, 28.42, 28.26, 28.22, 28.44, 28.2, 28.16, 28.47, 28.58, 28.55, 27.77, 28.66, 28.62, 28.68, 28.6, 26.79, 28.83, 28.9, 28.88, 28.95
	28.93, 28.91, 29.03, 29.14, 29.0, 28.99, 29.23, 29.18, 29.16, 29.24, 29.29, 29.31, 29.3, 29.15, 28.86, 28.82, 26.61, 29.52, 29.39, 29.59
	29.61, 29.57, 29.55, 29.63, 29.66, 29.72, 29.76, 29.7, 29.79, 29.81, 29.84, 30.04, 30.02, 30.19, 30.15, 30.08, 30.2, 29.93, 29.85, 29.8
	29.62, 30.22, 30.23, 30.29, 30.28, 30.26, 30.33, 30.41, 30.36, 30.31, 30.21, 30.47, 30.45, 30.53, 30.67, 30.72, 30.62, 30.5, 30.49, 30.42
	30.88, 30.84, 30.83, 30.82, 30.74, 30.93, 30.91, 31.06, 31.14, 31.22, 31.2, 31.16, 31.15, 31.08, 31.26, 31.33, 31.29, 31.38, 31.25, 30.89
	31.47, 31.5, 31.53, 31.56, 31.52, 31.42, 31.67, 31.65, 31.76, 31.7, 31.82, 31.69, 31.93, 31.91, 31.87, 31.61, 32.14, 32.15, 32.2, 32.07
	31.97, 31.95, 31.4, 32.3, 32.39, 32.44, 32.41, 32.36, 32.63, 32.69, 32.56, 32.75, 32.71, 32.46, 32.85, 32.97, 32.95, 32.99, 32.87, 32.83
	33.05, 33.09, 33.1, 33.03, 33.0, 33.43, 33.33, 33.44, 33.25, 33.24, 33.47, 33.22, 33.6, 33.67, 33.62, 33.56, 33.11, 32.81, 32.29, 33.77
	33.78, 33.83, 33.86, 33.84, 33.95, 33.93, 33.98, 33.87, 34.03, 34.11, 34.06, 34.17, 34.3, 34.38, 34.26, 34.42, 34.18, 34.53, 34.55, 34.49
	34.6, 34.59, 34.44, 34.0, 33.8, 34.63, 34.72, 34.67, 34.77, 34.73, 34.83, 34.84, 34.82, 34.93, 34.92, 34.91, 34.98, 34.96, 34.94, 34.78
	34.65, 33.73, 29.32, 25.23

Figure 4.3.8: Post-order traversal for temperature

Thus, the program will have the same flow also when the user's input is 3. The program will display " You have chosen binary search tree for thermostat".

```
---------------------------------------
 🌳 Binary Search Tree Program 🌳
---------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🎚️
[4] Binary search tree for search sensor
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*

Enter your choice: 3
You have chosen binary search tree for thermostat.
```

Figure 4.3.9: Main tree program when users choose 3

```
Building tree with these elements:
 [27, 28, 28, 20, 30, 23, 20, 21, 21, 27, 19, 25, 30, 19, 28, 23, 28, 19, 22

 +------------------+-------------------------------------------------------+
| Traversal Type   | Nodes                                                 |
+==================+=======================================================+
| In-order         | 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30 |
+------------------+-------------------------------------------------------+
| Pre-order        | 27, 20, 19, 18, 23, 21, 22, 25, 24, 26, 28, 30, 29 |
+------------------+-------------------------------------------------------+
| Post-order       | 18, 19, 22, 21, 24, 26, 25, 23, 20, 29, 30, 28, 27 |
+------------------+-------------------------------------------------------+
```

Figure 4.3.10: Three traversal for thermostat

More than that, the program also had a searching function which checks the location sensor (light, temperature and thermostat). The user needs to insert 4 when the system asks the user “Enter your choice” at binary search main tree program and enter the sensor available (light, temperature and thermostat). After that, the users need to enter the location search. The program will search the location of the sensor using binary search tree search. Then, the program will display location users enter found or not found in the sensor data. If the type of sensor is not found, the program will display “Your chosen sensor is not found in the Iot Home Monitoring System ”. Then, back to the main tree program.

```
--------------------------------------
 🌳 Binary Search Tree Program 🌳
--------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🚻
[4] Search for sensor location
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*

Enter your choice: 4
You have chosen to search for a sensor location.
Enter sensor type (light, temperature, or thermostat): light
Enter the location to search: bedroom

Building tree with these elements:
 ['kitchen', 'kitchen', 'kitchen', 'bathroom', 'garage', 'living room', 'kitc

Location 'bedroom' found in the light sensor data.
```

Figure 4.3.11: Users search sensor ‘light’ and location ‘bedroom’

```
Enter your choice: 4
You have chosen to search for a sensor location.
Enter sensor type (light, temperature, or thermostat): light
Enter the location to search: tandas

Building tree with these elements:
 ['kitchen', 'kitchen', 'kitchen', 'bathroom', 'garage', 'living room', 'kitchen', 'kitche

Location 'tandas' not found in the light sensor data.

--------------------------------------
 🌳 Binary Search Tree Program 🌳
--------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🚻
[4] Search for sensor location
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*

Enter your choice: 4
You have chosen to search for a sensor location.
Enter sensor type (light, temperature, or thermostat): humidity
The sensor you chose is not found in the IoT Home Monitoring System.
```

Figure 4.3.12: User search location and sensor not in data

After that, if the users choose other than 1/2/3/4/5, the program will display “Invalid choice. Please try again.” and back to the main tree program. Lastly, if the users choose 5, the program will display “Exiting tree program. See you again!👋” and back to the main page program.

```
---------------------------------------
 🌳 Binary Search Tree Program 🌳
---------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🚦
[4] Binary search tree for search sensor
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*

Enter your choice: 6
Invalid choice. Please try again.
```

Figure 4.3.13: User choose other than 1,2,3,4 & 5

```
---------------------------------------
 🌳 Binary Search Tree Program 🌳
---------------------------------------
Please choose one of your interests:
[1] Binary search tree for light intensity💡
[2] Binary search tree for temperature🌡️
[3] Binary search tree for thermostat🚦
[4] Binary search tree for search sensor
[5] Back to home page
*Traversal includes in-order, pre-order, and post-order*

Enter your choice: 5
Exiting three program. See you again!👋
```

Figure 4.3.14: Main tree program when users choose 5

4.4 GRAPH

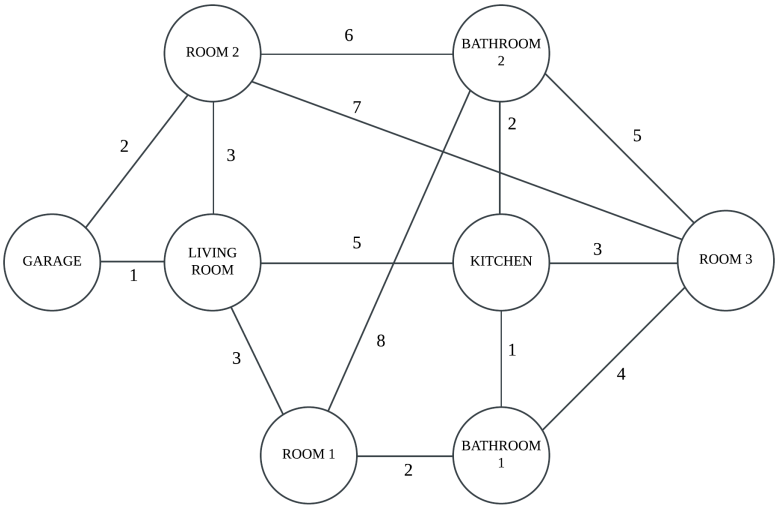


Figure 4.4.1 shows the graph representing the connection of devices inside the home

NODE NUMBER	LAYOUT
0	Garage
1	Living Room
2	Kitchen
3	Bathroom
4	Bedroom
5	Bathroom
6	Bedroom
7	Bedroom

Table 4.4 shows the node number that represents the layout of the house

Based on figure 4.4.1, it is an undirected graph layout of a home with smart devices installed. There are nine nodes and 14 vertices. Each of the nodes represents the layout of the home while the edges on each connected node represent the sensor device's connection to each other and the weight of the edges represents the speed of the connection from one layout to another layout. This program will show how the graph helps users to find unweighted paths using breadth-first search and depth-first search. In this program, the user will be able to find the shortest path with least weight using Prim's, Kruskal and Dijkstra's algorithm. As for the table 4.4, it represents the layout of the home using numbering to represent the node to ease the usage of the program.

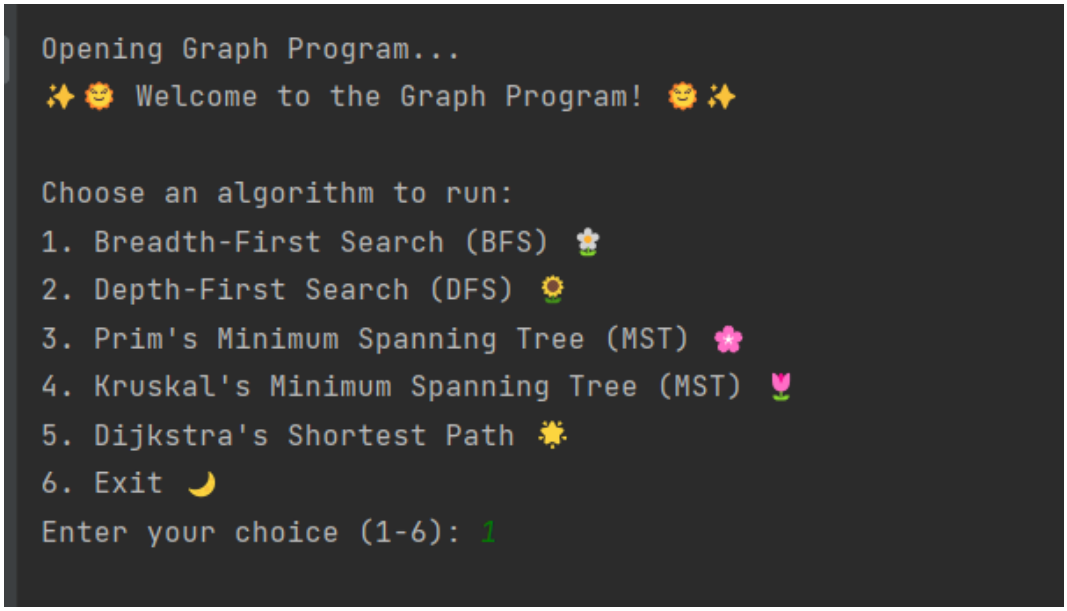


Figure 4.4.2 shows the output display of the graph main menu

From the graph menu above, it shows that five different graph algorithms are displayed in options. Option 1 will display Breadth-first search, option 2 will display Depth-first search, option 3 will display Prim's algorithm, option 4 will display Kruskal's algorithm and option 5 will display Dijkstra's algorithm. In addition, option 6 will exit the graph program and take the user back to the main menu.

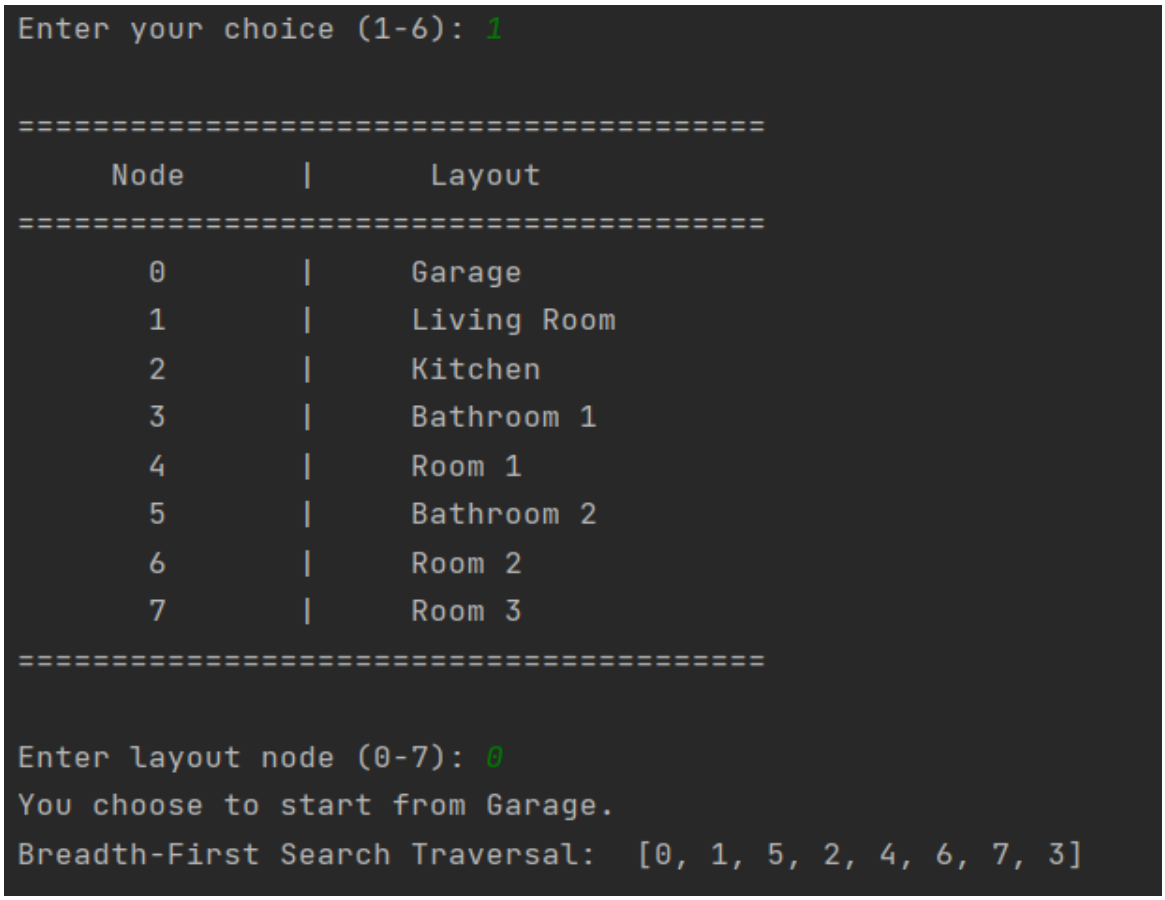


Figure 4.4.3 shows the display of breadth first search

When the user chooses option 1, the layout will display in a table and the user will be asked to enter the layout node. Then, it will display which layout user decides to start and the path using breadth-first search will display. In this demo, when the user chooses 0, it displays the garage and the breadth-first search for initial 0 is 0,1,5,2,4,6,7,3.

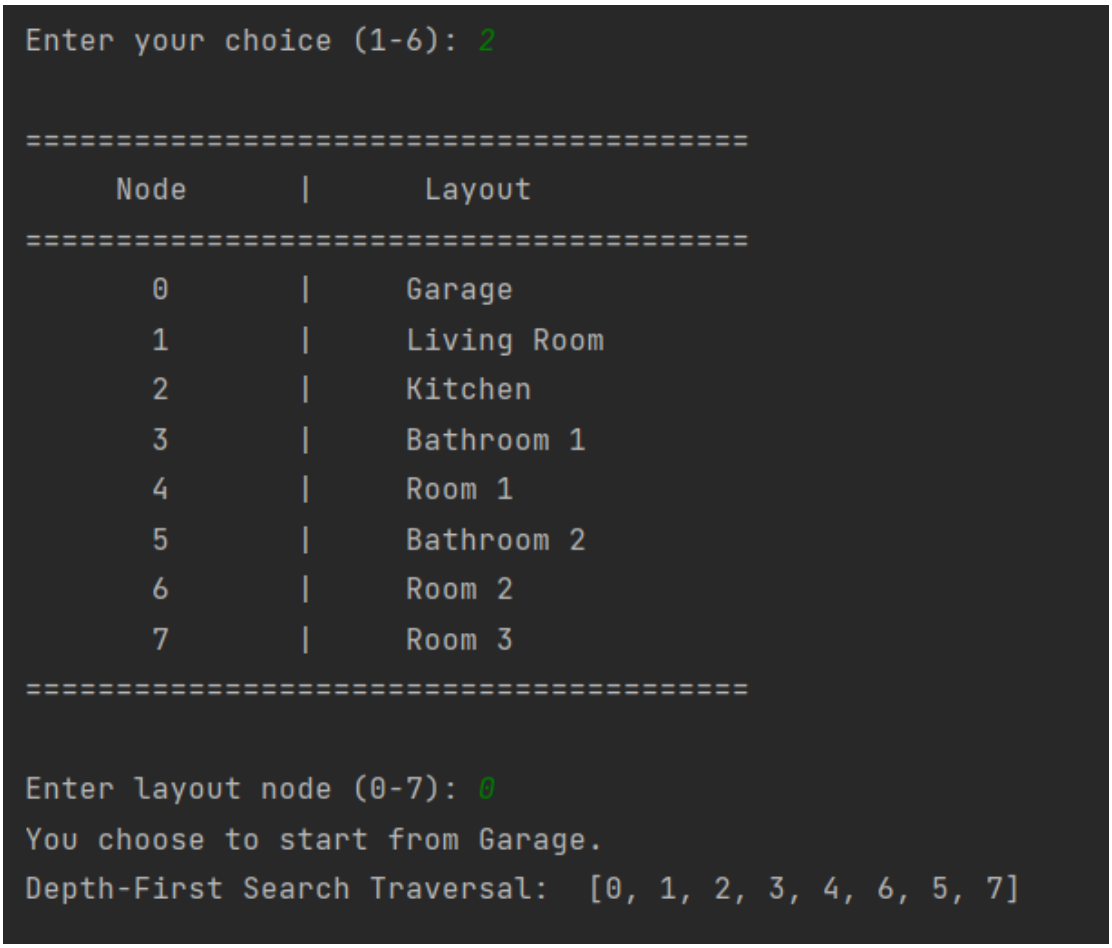


Figure 4.4.4 shows the display of depth-first search

When the user chooses option 2, the layout will display in a table and the user will be asked to enter the layout node. Then, it will display which layout user decides as starting node and the path using depth-first search will display. In this demo, when the user chooses 0, it displays the garage and the depth-first search for initial 0 is 0,1,2,3,4,6,5,7.

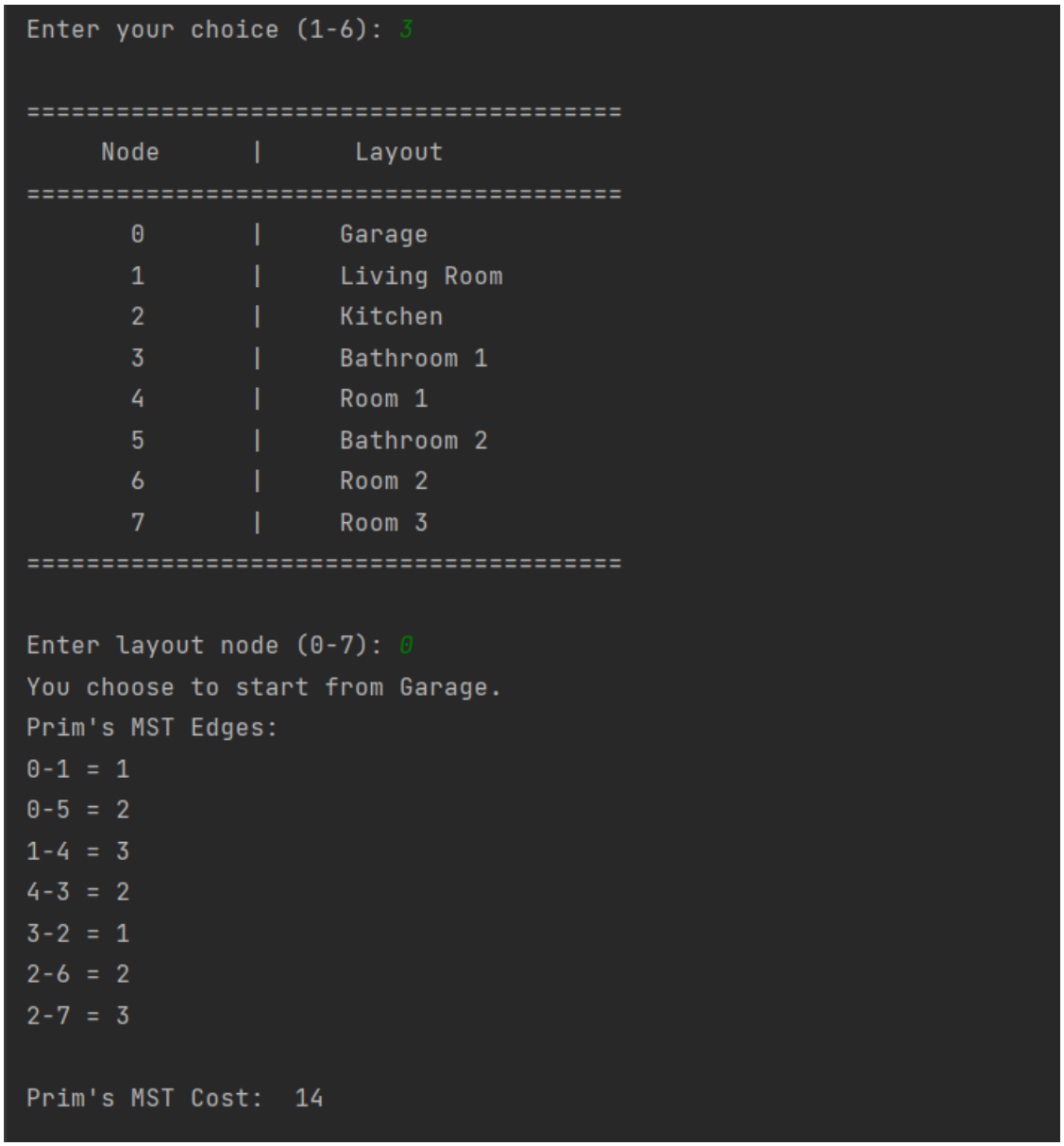


Figure 4.4.5 shows the display of Prim's algorithm

Next, when the user chooses option 3, the program displays the layout table and the user is asked to enter the layout node. Once the user has chosen the starting node, the program will display the path of connected nodes along with the weightage of the edge. Then, the minimal cost of prim's minimum spanning tree will be displayed. By referring to the demo above, when the user chooses 0 as the starting node, the programs list down all the least weight edges. The connections show 0-1, 0-5, 1-4, 4-3, 3-2, 2-6, 2-7 as the least weightage edges with 0 as starting node. The total minimal cost is 14.

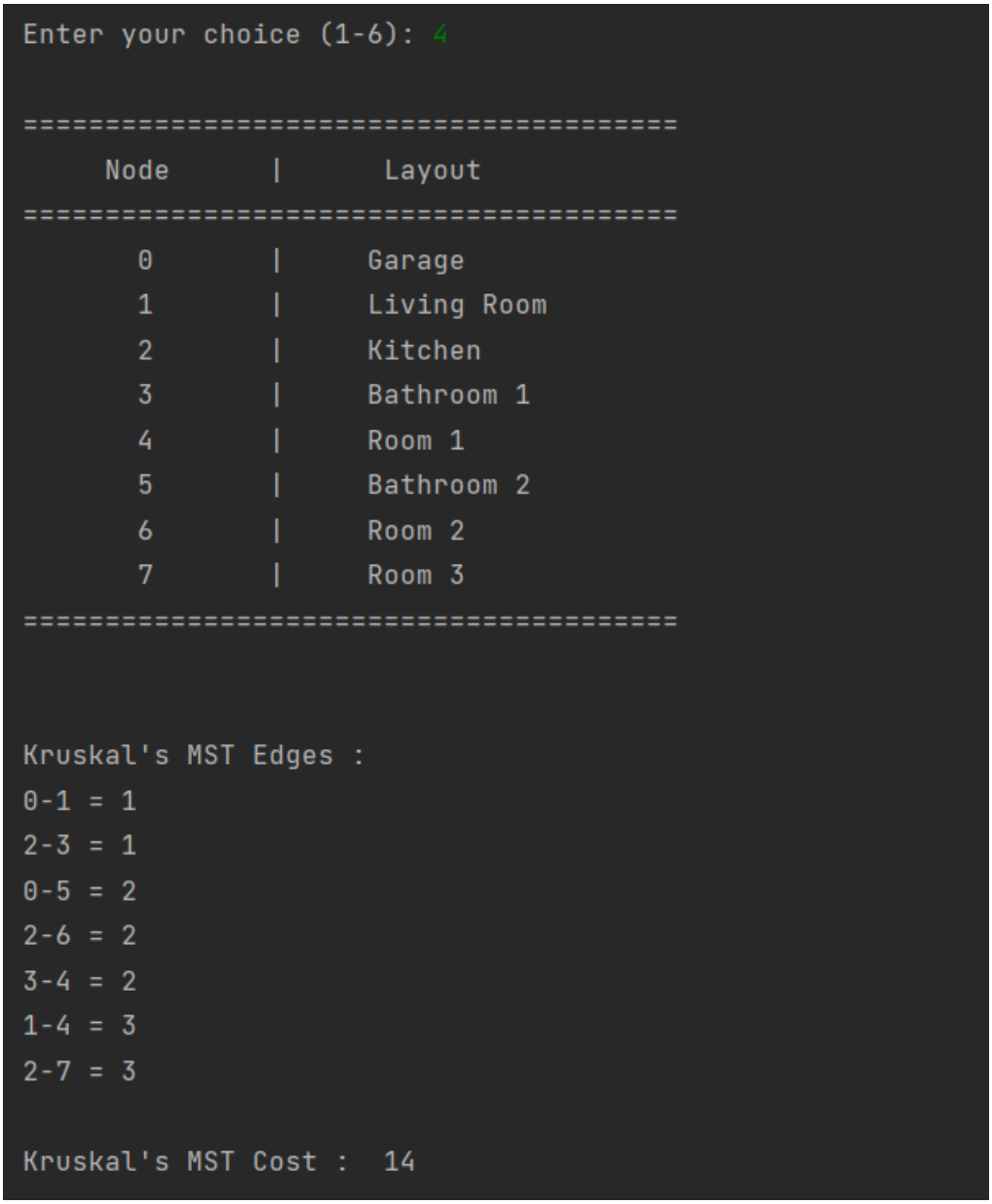


Figure 4.4.6 shows the display of Kruskal's algorithm

When the user chooses option 4, the layout table will appear and the program will directly list down all the node connections along with the least weightage edges and the minimal cost will show. In the demo, Kruskal's algorithm list down all the connected nodes of 0-1, 2-3, 0-5, 2-6, 3-4, 1-4 2-7 starting with the edges that has the least weight to the most weight and the total minimal cost is 14.

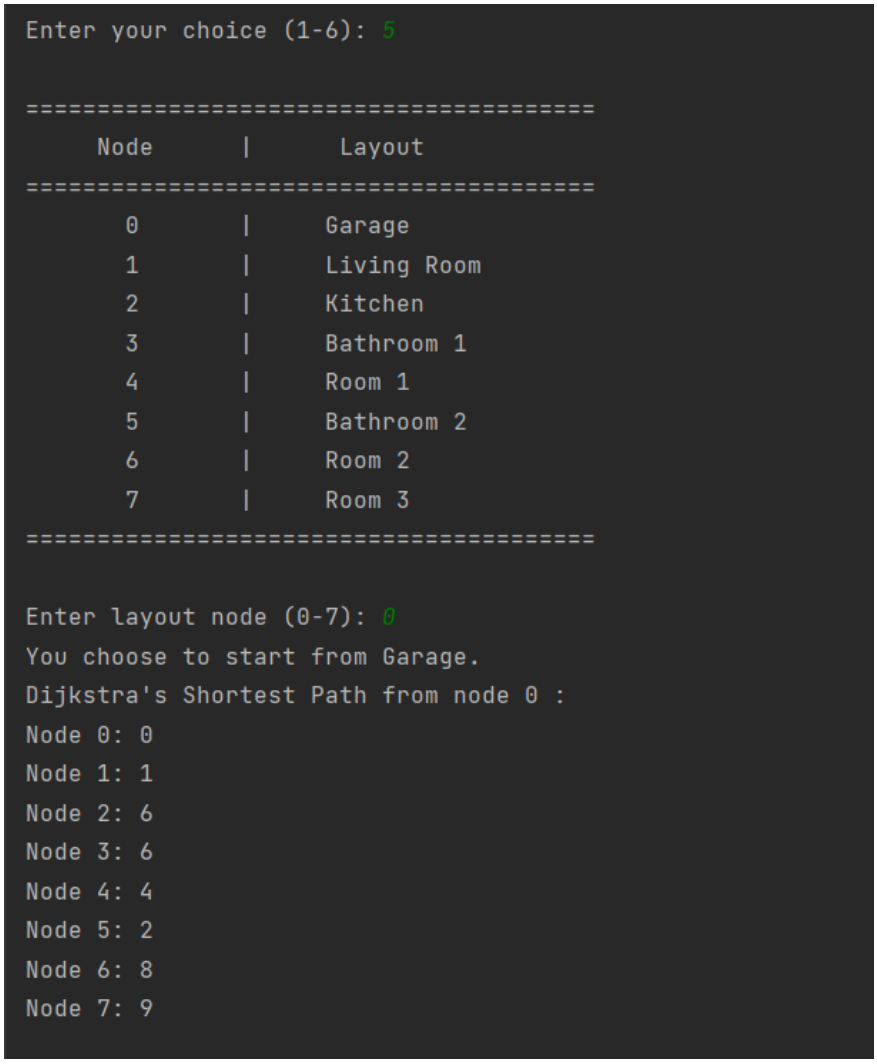


Figure 4.4.7 shows the display of Dijkstra's algorithm

Based on the figure, when the user chooses option 5, the layout table will appear and once again, the user will be asked to insert the layout node as the initial vertex then the program will display the list of the weight from the initial node to each other node. By referring to the demo, when the user choose 0 as the initial node, the result output shows the weightage of initial node to each other node with the shortest path.

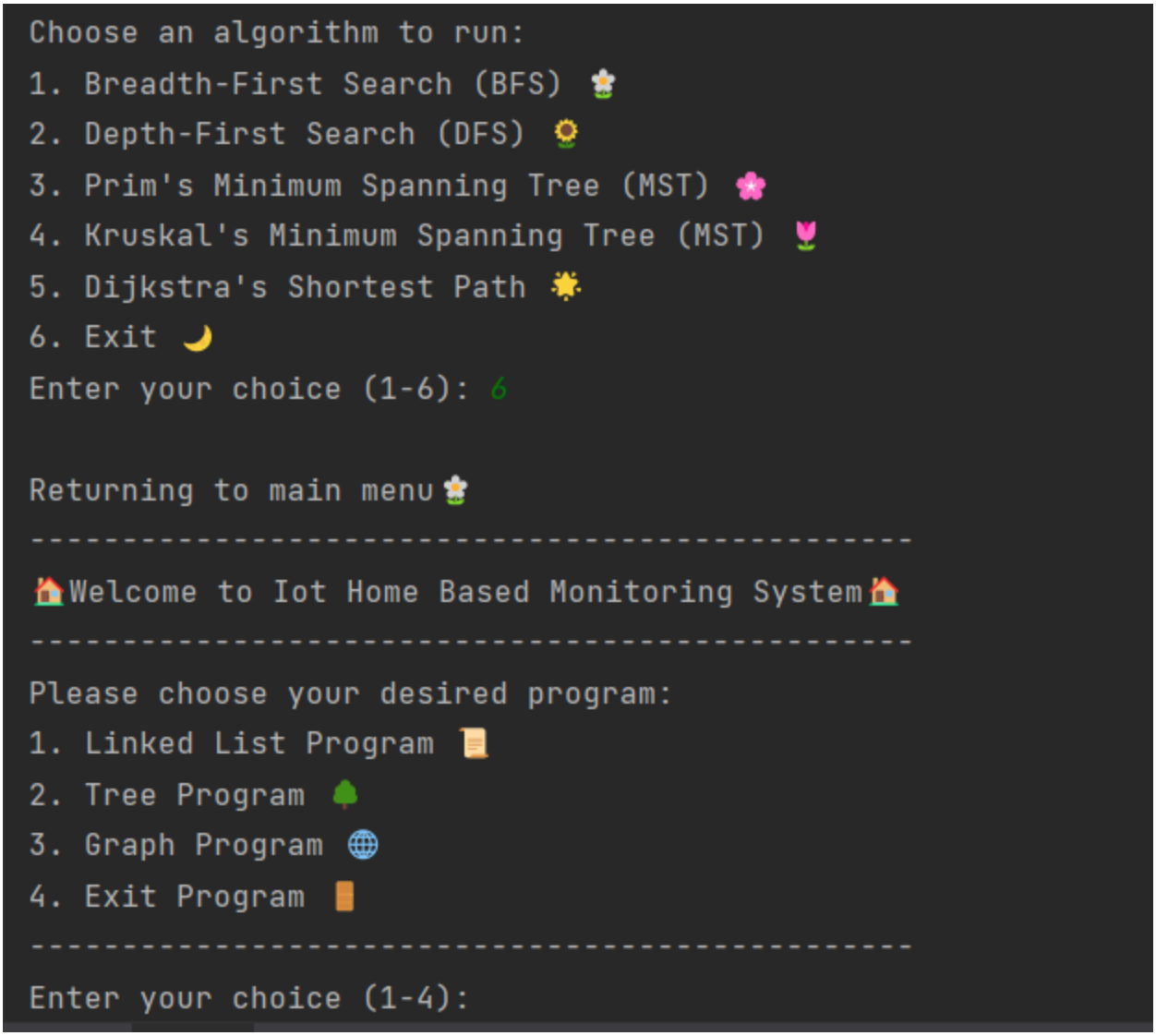


Figure 4.4.8 shows the display of the result of the exit option.

Based on the demo, when the user chooses option 6, the program will display “Returning to main menu” before taking the user back to the main menu.

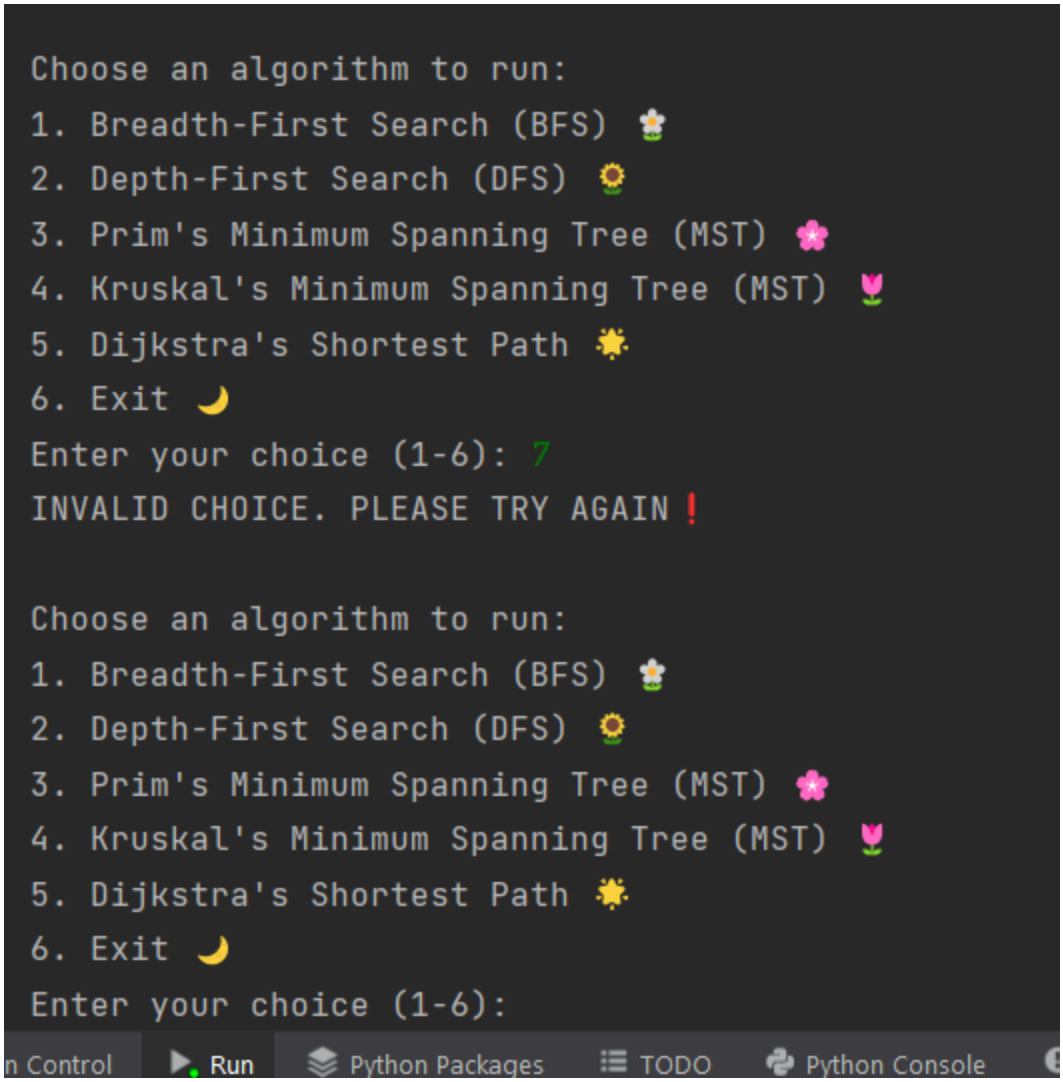


Figure 4.4.9 shows the display when insert incorrect number

When the user inserts input not in between 1 until 6, the program will display “INVALID CHOICE. PLEASE TRY AGAIN”. After that, the user will be taken back to the graph menu to insert the correct input.

5. CONCLUSION

Data structures such as linked lists, trees, and graphs play an important role in solving engineering problems by providing efficient solutions for dynamic data management, complex relationship analysis, and process flow optimization in various applications. The first objective is to develop a database system to add, remove, and sort all the smart home devices that have been installed using a linked list. It can be concluded that the objective that has been achieved by adding device data is that a new node is added to the linked list, which represents the smart device data. This operation is performed by assigning new device data and connecting it to an existing node. Next, removing device data means that data can be removed from the linked list by adjusting the node pointer before and after the target node.

Then, sorting device data is that the data in the linked list is successfully sorted based on parameters such as Light_Intensity using Bubble Sort. To improve the efficiency and usability of linked list systems in smart home applications, an improvement that can be considered is to replace bubble sort with algorithms such as merge sort or quick sort to improve performance, especially when there are many devices. This is because merge sort or quick sort is a more efficient sorting algorithm compared to bubble sort.

Next, we can conclude that the second objective has been achieved. The use of Binary Search Trees (BST) algorithm provided a systematic and efficient solution to the challenges of managing IoT devices in smart homes. By organizing data hierarchically and enabling fast retrieval, BST improved the overall performance and usability of the system. This allowed for quick and accurate searching of specific devices or reading, even as the number of devices and location increased. The traversal methods further enhance the system's flexibility by enabling users to access data in various order based on their needs. Additionally, the BST allowed users to locate specific sensors or readings efficiently, reducing the time and effort required to manage the devices. Future work could explore the integration of self-balancing to further optimize the system for large datasets and more complex IoT environments.

Last but not least, it can be concluded that the third objective has been achieved. A topology graph is created based on the input data used for the mini project. By referring to the graph, the program is able to perform breadth-first search traversal and depth-first search traversal. Apart from that, the graph program is also able to perform greedy methods that can be divided into three; Prim's algorithm, Kruskal algorithm and Dijkstra's algorithm. Total minimal cost of the nodes can be found using Prim's and Kruskal while shortest path for each node can be found by using Dijkstra's algorithm.